

Fast Marching Methods*

J. A. Sethian[†]

Abstract. Fast Marching Methods are numerical schemes for computing solutions to the nonlinear Eikonal equation and related static Hamilton–Jacobi equations. Based on entropy-satisfying upwind schemes and fast sorting techniques, they yield consistent, accurate, and highly efficient algorithms. They are optimal in the sense that the computational complexity of the algorithms is $O(N \log N)$, where N is the total number of points in the domain. The schemes are of use in a variety of applications, including problems in shape offsetting, computing distances from complex curves and surfaces, shape-from-shading, photolithographic development, computing first arrivals in seismic travel times, construction of shortest geodesics on surfaces, optimal path planning around obstacles, and visibility and reflection calculations. In this paper, we review the development of these techniques, including the theoretical and numerical underpinnings; provide details of the computational schemes, including higher order versions; and demonstrate the techniques in a collection of different areas.

Key words. Fast Marching Methods, Eikonal equation, Hamilton–Jacobi equations

AMS subject classifications.

PII. S0036144598347059

1. Introduction to Fast Marching Methods. Fast Marching Methods are computational techniques that approximate the solution to nonlinear Eikonal equations of the form

$$(1) \quad |\nabla u(x)| = F(x) \text{ in } \Omega, \quad F(x) > 0,$$

$$u = g(x) \text{ on } \Gamma,$$

where Ω is a domain in R^2 or R^3 .¹ Here, the right-hand side, $F(x) > 0$, is typically supplied as known input to the equation, as is the boundary condition that u equal a known function $g(x)$ given along a prescribed curve or surface Γ in Ω .

Equation (1) is part of a broader class of Hamilton–Jacobi equations of the form

$$(2) \quad H(u_x, u_y, u_z, x, y, z) = 0.$$

In the case of the Eikonal equation, the function H reduces to $H = |\nabla u(x)| - F(x)$.

*Received by the editors November 9, 1998; accepted for publication (in revised form) November 13, 1998; published electronically April 23, 1999. This work was supported in part by the Applied Mathematical Sciences subprogram of the Office of Energy Research, U.S. Department of Energy, under contract DE-AC03-76SF00098, and the Office of Naval Research under grant FDN00014-96-1-0381.

<http://www.siam.org/journals/sirev/41-2/34705.html>

[†]Department of Mathematics, University of California, Berkeley, California 94720 (sethian@math.berkeley.edu).

¹The theory and Fast Marching Methods algorithms presented in this paper hold for R^n ; for notational reasons, we focus on R^2 and R^3 .

As discussed below, one of the main difficulties in solving these equations is that the solution need not be differentiable, even with smooth boundary data. This nondifferentiability is intimately connected to the notion of appropriate weak solutions. Our goal is to devise numerical techniques that naturally account for this nondifferentiability by constructing accurate and efficient approximation schemes that admit physically correct nonsmooth solutions. The main techniques we will use are Fast Marching Methods, introduced in [27]. These consistent and highly efficient techniques are based on two key components. First, by exploiting upwind “viscosity schemes,” they automatically select solutions that include nondifferentiability in natural ways. Second, by coupling the causality of these schemes to fast sorting methods borrowed from discrete network problems, they become extremely efficient computationally; the complexity of these algorithms is $O(N \log N)$, where N is the total number of points in the domain Ω .

There is a large collection of applications that require the solution of the nonlinear Eikonal equation. To begin, imagine the problem of accurately computing the distance to a curve or surface. By letting the right-hand side F equal unity, the level curves $u = C$ of the Eikonal equation give the set of all points located a distance C from the boundary curve; this can be seen by noting that the gradient $|\nabla u|$ must be orthogonal to these level curves, and that the gradient has length 1. Another way to state this problem is to imagine a disturbance propagating with unit speed away from the initial curve, and then to compute the “first arrival time” at each point in the domain Ω . Points where the solution is nondifferentiable are locations where two points on the boundary curve are the same distance away. Performing this same calculation on surfaces and tracking backwards orthogonally to equiarrival curves allows one to compute shortest paths on manifolds.

Extending the problem by changing the right-hand side to a nonconstant F corresponds to computing the distance function in a nonuniform metric; here the “arrival times” are slowed or sped up by the variation in F . Two examples include the calculation of first arrivals in seismic travel times, in which the “slowness” function F corresponds to the reciprocal of the velocity that varies depending on the type of rock, and the development process in photolithography, in which the resistive strength of a material is differentially altered through optical processes and the material is then exposed to an etching beam that removes the weaker material.

More drastic alterations of the right-hand side (that is, setting $F = \infty$ in certain subsets of Ω) yield equations for optimal path navigation, in which an infinite value for F corresponds to an impenetrable obstacle that must be circumnavigated. Extending the dimension from pure physical space (from R^2 or R^3) to include rotational effects allows one to include additional degrees of freedom generated by moving arms in the navigation process.

The outline of this paper is as follows. We begin by discussing the ideas of weak solutions and viscosity solutions, followed by numerical approximations and upwind schemes. We then present the Fast Marching Method, first in orthogonal coordinate systems, and then in a triangulated unstructured mesh setting. Here, we discuss in some detail the connection of these techniques to both network path algorithms and level set methods. After extending the Fast Marching Method to higher order, we end with a collection of applications.

Before we start, we point out a slightly confusing issue that is at the heart of the efficiency of Fast Marching Methods. The nonlinear Eikonal equation (1) and the static Hamilton–Jacobi equation (2) are *boundary value problems*, but the notion of “first arrival times” sounds more like an *initial value problem*, in which information

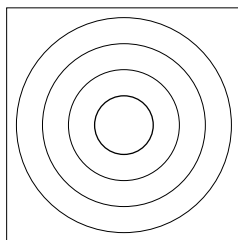


Fig. 1 Distance function solution to Eikonal equation $|\nabla u| = 1$.

propagates outwards from the boundary/initial data. In fact, whereas the equations are true boundary value problems, the success of Fast Marching Methods comes from building numerical schemes that allow one to efficiently construct the solution outward from the boundary data.

The majority of this review is taken from the original work on Fast Marching Methods [27] and two recent books on the subject [30, 31]. We refer the interested reader to these resources for many computational schemes that can exploit Fast Marching Methods, as well as many more applications and examples.

2. The Eikonal and Static Hamilton–Jacobi Equations.

2.1. Fundamental Equations and Nondifferentiability. We begin by returning to the example of the simple Eikonal equation

$$|\nabla u| = F(x, y),$$

with $F = 1$. One of the subtleties of this equation is that the solution may be nondifferentiable, even with smooth initial data. As a simple illustration, consider the case of

$$(3) \quad |\nabla u| = 1 \text{ outside the unit circle,} \quad u = 0 \text{ on the unit circle.}$$

It is readily checked that the function $u(x, y) = (x^2 + y^2)^{1/2} - 1$ corresponding to the distance function from the unit circle solves the given Eikonal equation. Indeed, specifying the right-hand side $F(x) = 1$ and providing zero as the boundary condition implies that the solution is just the distance to the initial curve Γ . The solution is shown as a family of concentric circles in Figure 1. It is, of course, easy to check that the solution is everywhere differentiable.

In contrast, now consider boundary data given by a nonconvex curve, and suppose we try to find the distance from any point in the domain to this curve. There are two ways we can proceed.

First, suppose we put a particle at each point of the boundary curve and move these particles away from the curve in a normal direction with unit speed. At any time C , the position of these particles gives a set of all points a “distance” C away from the boundary curve. Figure 2a illustrates this approach above a particular nonconvex curve.

We note that this construction creates a curve that crosses itself. This “swallowtail” solution is multivalued; some points are reached by normals emanating from more than one point on the boundary.

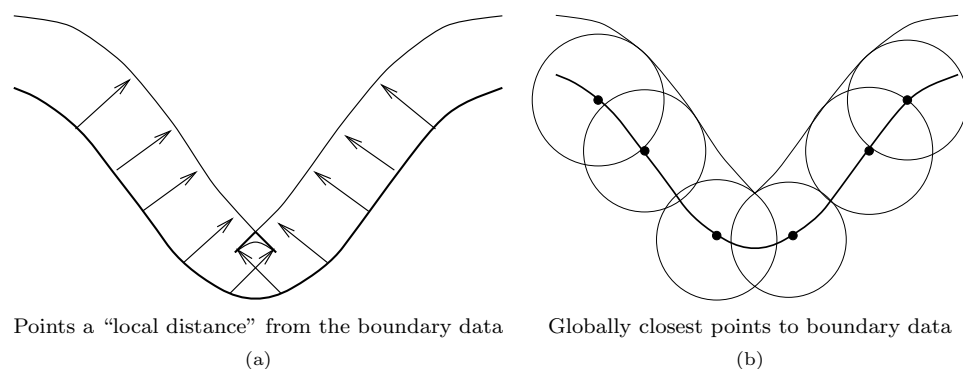


Fig. 2 Possible solutions to Eikonal equation $|\nabla u| = 1$.

A different approach comes from drawing the set of all points located a given distance away. One way to build this solution (shown on the right) is through a Huygens principle construction. The solution is developed by imagining wavefronts emanating with unit speed from each point of the boundary data. The envelope of these wavefronts always corresponds to the “first arrivals” and automatically produces the solution given in Figure 2b. We note that there will be a vertical ridge along which two points on the boundary curve are the same distance away, as suggested in Figure 2b. Along this ridge, the solution is nondifferentiable and the gradient ∇u is not defined.

Both of the above constructions can be thought of as solutions to the Eikonal equation. However, the solution that we want, corresponding to the shortest distance or “first arrival,” is the one obtained through the Huygens construction. Another way to obtain this solution is through the notion of an entropy condition. As defined in [23] and [24], we imagine the boundary curve as a source for a propagating flame, and the expanding flame satisfies the requirement that once a point in the domain is ignited by the expanding front, it stays burnt. This construction yields the entropy-satisfying Huygens construction given in Figure 2b.

Yet another way to obtain this entropy-satisfying Huygens construction comes from adding a smoothing term to the equation. Consider the associated “viscous” partial differential equation given by

$$(4) \quad |\nabla u(x)| = F(x) + \epsilon \nabla^2 u.$$

It can be shown that the viscous term $\epsilon \nabla^2 u$ acts to smooth out sharp corners in the solution and guarantees that the solution stays smooth in the entire domain Ω (see [24, 30]). As ϵ goes to zero, the solution converges to the first arrival solution given in Figure 2b.

Thus, our goal is to build numerical methods that automatically extract this viscous limit. Before doing so, we note that the formal way of defining this viscous limit is through the idea of viscosity solutions for time-dependent Hamilton–Jacobi equations. Rather than define the solution as the viscous limit, one instead analyzes the behavior of potential solutions when measured against possible test functions. Crandall and Lions [8] have developed the theory of viscosity solutions for time-dependent Hamilton–Jacobi equations. Briefly, following the definitions in [8], they define a viscosity solution as follows.

DEFINITION. A function u is said to be a *viscosity solution* of (2) for all smooth test functions v ,

1. if $u - v$ has a local maximum at a point (x_o, t_o) , then

$$(5) \quad v_t(x_o, t_o) + H(Dv(x_o, t_o), x_o) \leq 0;$$

2. if $u - v$ has a local minimum at a point (x_o, t_o) , then

$$(6) \quad v_t(x_o, t_o) + H(Dv(x_o, t_o), x_o) \geq 0.$$

Note that nowhere in this definition is the viscosity solution u differentiated; everything is done in terms of the test function v . This is done so that one can now use the usual trick of integration by parts and move all the derivatives onto the test function in exchange for some boundary conditions.

Using this definition, one can show that (i) if u is a smooth solution of the Hamilton–Jacobi equation, then it is a viscosity solution, (ii) if a viscosity solution u is differentiable at some point, then it satisfies the Hamilton–Jacobi equation at that point, and (iii) the viscosity solution is unique, given appropriate initial conditions. Finally, one then proves that the solution produced by taking the limit of the smooth solutions u_ϵ as ϵ vanishes is indeed this viscosity solution.

We shall prove none of these statements here. Precise statements and proofs may be found in [6, 8, 7]. The salient point is that for both time-dependent and static (e.g., the Eikonal equation) Hamilton–Jacobi equations, the viscosity solution can be defined in a way that does not require differentiation, and can be proven to be the unique viscous limit of the smoothed Hamilton–Jacobi equation.

Our goal is to develop numerical approximations that correctly select this viscous limit. As proposed in [25], the fact that the entropy condition is similar to the one for hyperbolic conservation laws suggests the use of the numerical methodologies associated with the hyperbolic equation.

2.2. Upwind Schemes and Numerical Approximations.

2.2.1. Upwind Schemes and Numerical Quadrature. As motivation for the use of upwind schemes for approximating the gradient operator, consider the one-dimensional Eikonal equation given by

$$\sqrt{u_x^2} = F(x), \quad u(0) = 0.$$

Here, the right-hand side $F(x) > 0$ is given, and the goal is to construct $u(x)$ away from the boundary condition that $u(0) = 0$. We note immediately that the solution to this problem is not unique; if $v(x)$ solves the problem, then so does $-v(x)$. Hence, we further restrict ourselves to nonnegative solutions u .

We can imagine building the solution “outwards” along the positive and negative x -axis from the origin by solving each problem separately. We consider the ordinary differential equations

$$\frac{du}{dx} = F(x), \quad u(0) = 0, \quad x \geq 0,$$

$$\frac{du}{dx} = -F(x), \quad u(0) = 0, \quad x \leq 0.$$

Since the right-hand side is only a function of x , we are essentially performing numerical quadrature. Using the standard finite difference notion that $u_i \approx u(i\Delta x)$, and

$F_i = F(i\Delta x)$, we can approximate each of these two solutions using Euler's method, namely,

$$\frac{u_{i+1} - u_i}{\Delta x} = F_i, \quad i > 0,$$

$$\frac{u_i - u_{i-1}}{\Delta x} = -F_i, \quad i \leq 0,$$

where $u_0 = 0$. These are upwind schemes: we are computing derivatives using points "upwind" or towards the boundary condition. In other words, each ordinary differential equation is solved away from the boundary condition.

2.2.2. Upwind Schemes and Evolving Interfaces. Further motivation for approximating the gradient using upwind differences comes from the instructive example of a curve evolving in time whose position can always be described as the graph of a function. Consider an initial front given by the graph of $g(x)$, with g and g' periodic on $[0, 1]$, and suppose that the front (i) propagates with speed S in its normal direction and (ii) remains a function for all time. Let ψ be the height of the propagating function at time t ; thus $\psi(x, 0) = g(x)$. One can show (see [25]) that the equation of motion for the changing height is given by

$$(7) \quad \psi_t = S(1 + \psi_x^2)^{1/2}.$$

As an example, with speed $S = 1$, consider the initial value problem

$$(8) \quad \psi_t = (1 + \psi_x^2)^{1/2}, \quad \psi(x, 0) = g(x) = \begin{cases} 1/2 - x, & x \leq 1/2, \\ x - 1/2, & x > 1/2. \end{cases}$$

The initial front is a "V" formed by rays meeting at $(1/2, 0)$. By the entropy condition and Huygens principle construction, the solution at any time t is the set of all points located a distance t from the initial V. Our goal is to show that the choice of numerical approximations for the gradient term $(1 + \psi_x^2)^{1/2}$ has some subtlety.

One approach is to divide the interval $[0, 1]$ into $2M - 1$ points and to form the central difference approximation to the spatial derivative ψ_x in (8), namely,

$$(9) \quad \psi_t \approx \frac{\psi_i^{n+1} - \psi_i^n}{\Delta t} = \left[1 + \left[\frac{\psi_{i+1}^n - \psi_{i-1}^n}{2\Delta x} \right]^2 \right]^{1/2} = [1 + [D_i^{0x}\psi]^2]^{1/2},$$

where in the last expression we have used standard notation for the central difference.

Since $x_M = 1/2$, by symmetry we have that $\psi_{M+1} = \psi_{M-1}$; thus the right-hand side is 1. However, for all $x \neq 1/2$, ψ_t is correctly calculated to be $\sqrt{2}$, since the graph is linear on either side of the corner, and thus the central difference approximation is exact. Note that this has nothing to do with the size of the space step Δx or the time step Δt . No matter how small we make the numerical parameters, as long as we use an odd number of points, the approximation to ψ_t at $x = 1/2$ gets no better. It is simply due to the way in which the derivative ψ_x is approximated. In Figure 3 we show results using this scheme, with the time derivative ψ_t replaced by a forward difference scheme.

It is easy to see what has gone wrong. In the exact solution, $\psi_t = \sqrt{2}$ for all $x \neq 1/2$. This should also hold at $x = 1/2$, where the slope is not defined; the Huygens construction sets $\psi_t(x = 1/2, t)$ equal to $\lim_{x \rightarrow 1/2} \psi_t$. Unfortunately, the central difference approximation chooses a different (and, for our purpose, wrong) limiting solution. It sets the undefined slope ψ_x equal to the average of the left and right slopes. As the calculation progresses, this miscalculation of the slope propagates outwards from the spike as wild oscillations. Eventually, these oscillations cause blowup in the code. For details, see [30].

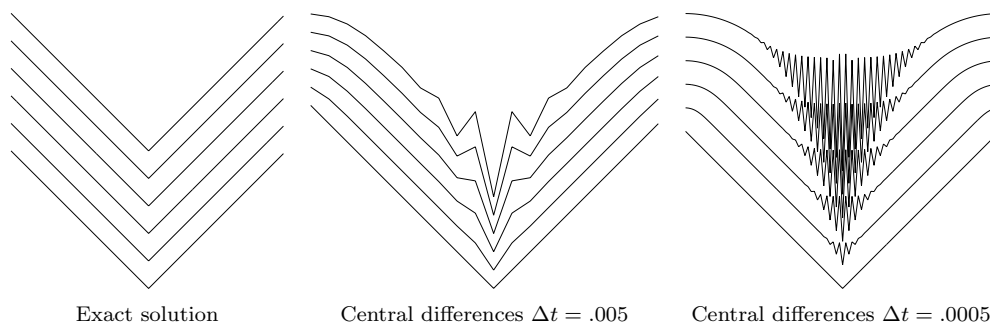


Fig. 3 Central difference approximation to gradient.

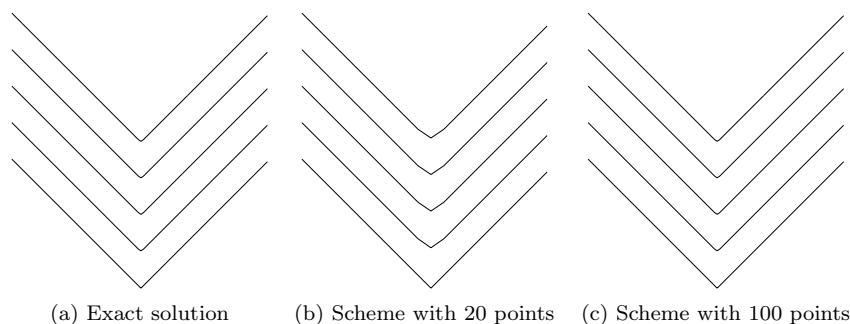


Fig. 4 Upwind, entropy-satisfying approximations to the gradient.

2.3. Schemes for Viscosity Solutions. Continuing with the example of an evolving interface, we now focus on the gradient term $(1 + \psi_x^2)$. Consider now the finite difference approximation introduced in [18], namely,

$$(10) \quad \psi_x^2 \approx (\max(D_i^{+x}\psi, 0))^2 + \min(D_i^{-x}\psi, 0)^2,$$

where again we have used standard finite difference notation:

$$(11) \quad D_i^{-x}\psi = \frac{\psi_i - \psi_{i-1}}{h}, \quad D_i^{+x}\psi = \frac{\psi_{i+1} - \psi_i}{h}.$$

Here, ψ_i is the value of ψ on a grid at the point ih with grid spacing h .

Equation (10) is an upwind scheme (see [30]); it chooses grid points in the approximation in terms of the direction of the flow of information. If we consider our propagating V curve from the example above, we see that a nonzero value is chosen at the symmetric point. In Figure 4, we show what happens if we use the scheme given in (10). The exact answer is shown, together with two simulations. The first uses the entropy-satisfying scheme with only 20 points (Figure 4b), the second (Figure 4c) with 100 points. In the first approximation, the entropy condition is satisfied, but the corner is somewhat smoothed due to the small number of points used. In the more refined calculation, the corner remains sharp, and the exact solution is very closely approximated. Thus we see that this scheme does a correct job of satisfying the entropy condition.

It is relatively straightforward to prove that this numerical scheme converges to the correct viscosity solution. Crandall and Lions [8] proved that consistent monotone

schemes must converge to the correct viscosity solution; this result parallels similar results for hyperbolic conservation laws (see, for example, [4, 15, 34]). Thus, we need only check that this scheme satisfies the necessary requirements for convergence to viscosity solutions. The operator is easily seen to be consistent, since it uses first-order finite difference operators, and hence a Taylor series expansion of the error can be shown to go to zero as the time and space steps are refined. Proving monotonicity for the most basic first-order scheme is also straightforward and is done by simply checking what happens to monotone data. One also can go back and check that the central difference scheme given earlier does not satisfy these requirements. We refer the interested reader to [8] and [31] for further comment about convergence and for additional schemes.

While a vast array of other upwind, entropy-satisfying schemes are available to approximate the gradient, for our purposes the above approximation (and one small variation) will be sufficient; details on other schemes may be found in [28, 30].

2.4. Approximations to the Eikonal Equation. We can now construct appropriate schemes for the Eikonal equation

$$(12) \quad |\nabla u(x, y, z)| = F(x, y, z).$$

Extending these ideas of upwind approximations for the gradient to multiple dimensions, we have the scheme

$$(13) \quad |\nabla u| \approx \left[\begin{array}{l} \max(D_{ijk}^{-x}u, 0)^2 + \min(D_{ijk}^{+x}u, 0)^2 \\ + \max(D_{ijk}^{-y}u, 0)^2 + \min(D_{ijk}^{+y}u, 0)^2 \\ + \max(D_{ijk}^{-z}u, 0)^2 + \min(D_{ijk}^{+z}u, 0)^2 \end{array} \right]^{1/2} = F_{ijk}.$$

The forward and backward operators D^{-y} , D^{+y} , D^{-z} , and D^{+z} in the other coordinate directions are similar to those defined earlier for the x direction.

A slightly different upwind scheme due to Godunov (see [20]), which will turn out to be more convenient, is given by

$$(14) \quad \left[\begin{array}{l} \max(D_{ijk}^{-x}u, -D_{ijk}^{+x}u, 0)^2 \\ + \max(D_{ijk}^{-y}u, -D_{ijk}^{+y}u, 0)^2 \\ + \max(D_{ijk}^{-z}u, -D_{ijk}^{+z}u, 0)^2 \end{array} \right]^{1/2} = F_{ijk},$$

where we use the same forward and backward operators D^{-} and D^{+} , and F_{ijk} is the slowness at the grid point ijk .

How might one solve (14)? One solution, given by Rouy and Tourin in [20], is through iteration. Consider a stencil of a grid point and its six neighbors, as shown in Figure 5. Observe that (14) is a piecewise quadratic equation for u_{ijk} , assuming that the neighboring grid values for u are given. Thus, one solution comes from updating the value of u at each grid point according to this quadratic until a solution is reached:

$$(15) \quad \begin{array}{l} \text{For iter}=1,n; \\ \quad \text{For } i, j, k=1, \text{Mesh} \\ \quad \quad \text{Solve Quadratic for } u_{ijk}^{\text{iter}+1}, \text{ given} \\ \quad \quad \quad u_{i-1,j,k}^{\text{iter}}, u_{i+1,j,k}^{\text{iter}}, u_{i,j-1,k}^{\text{iter}}, u_{i,j+1,k}^{\text{iter}}, u_{i,j,k-1}^{\text{iter}}, u_{i,j,k+1}^{\text{iter}} \\ \quad \text{EndFor} \\ \text{EndFor} \end{array}$$

If we assume that there are N points in each direction and that it takes roughly N steps to converge, then the operation count for this method is $O(N^4)$.

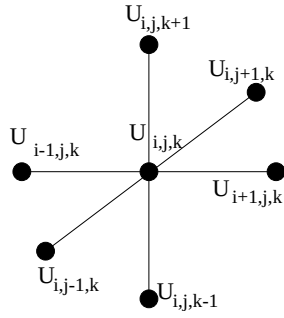


Fig. 5 Updating a grid point.

3. Fast Marching Methods. The key to Fast Marching Methods lies in the observation that the previous iteration contains a very specific causality relationship. In this section, we describe the Fast Marching Method; for details, see [27, 28, 30].

3.1. Causality. The central idea behind the Fast Marching Method is to systematically construct the solution in a “downwind” fashion to produce the solution u . We observe that the upwind difference structure of (14) means that information propagates “one way,” that is, from smaller values of u to larger values. Hence, the Fast Marching Algorithm rests on “solving” (14) by building the solution outwards from the smallest u value. In fact, this strategy is behind our quadrature view of upwinding described earlier; we can step the solution outwards from the boundary condition in a downwind direction. The algorithm is made fast by confining the “building zone” to a narrow band around the front; this approach is motivated by the narrow band technology introduced by Chopp [3], used in recovering shapes in images by Malladi, Sethian, and Vemuri [16], and analyzed extensively by Adalsteinsson and Sethian in [1]. The idea is to sweep the front ahead in a downwind fashion by considering a set of points in a narrow band around the existing front, and to march this narrow band forward, freezing the values of existing points and bringing new ones into the narrow band structure. The key is in the selection of *which* grid point in the narrow band to update.

Consider a two-dimensional version of the Eikonal equation, in which the boundary value is known at the origin; this is shown schematically in Figure 6. The black sphere at $u_{0,0}$ signifies a grid point where the value of u is known (in this case, the initial value), and the light grey spheres are grid points where the solution value is unknown.

We may start the algorithm by marching “downwind” from the known value, computing new values at each of the four neighboring grid points, as shown in Figure 7. This provides possible values for u at each grid point $u_{-1,0}, u_{1,0}, u_{0,-1}, u_{0,1}$; these values are shown as dark grey spheres in Figure 7.

Now, we would like to march downwind from these values given at the dark grey spheres, but we do not know which one to choose. The answer lies in the observation that the *smallest u value at these dark grey spheres must be correct*. Because of upwinding, no point can be affected by grid points containing larger values of u . Thus, we may freeze the value of u at this smallest dark grey sphere,² and proceed ahead with the algorithm; this is shown schematically in Figure 7.

²That is, we turn it into a black sphere and consider its value known.

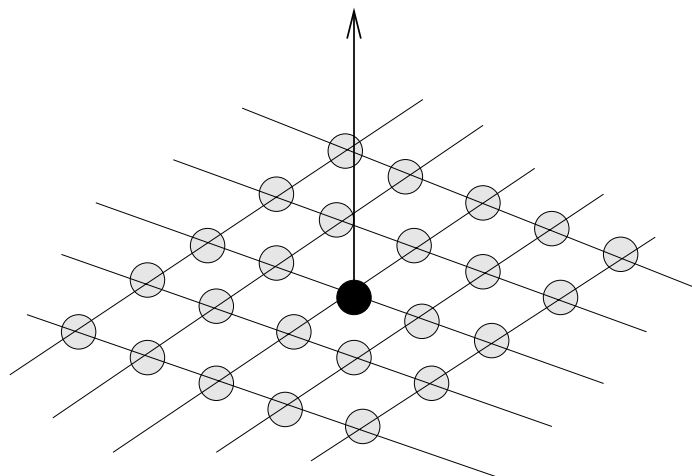


Fig. 6 Beginning of Fast Marching Method.

This algorithm works because the process of recomputing the u values at downwind neighboring points cannot yield a value smaller than any of the accepted points. Thus, we can march the solution outwards, always selecting the narrow band grid point with minimum trial value for u and readjusting downwind neighbors (see Figure 8).

Another way to look at this scheme is that each minimum trial value begins an application of the Huygens principle, and the expanding wavefront touches and updates all others. The speed of the algorithm comes from a heapsort technique to efficiently locate the smallest element in the set *Trial*.

Thus, the Fast Marching Method is as follows: First, tag points in the initial conditions as *Alive*. Then tag as *Close* all points one grid point away. Finally, tag as *Far* all other grid points. Then the loop is as follows.

1. Begin Loop: let *Trial* be the point in *Close* with the smallest value of u .
2. Tag as *Close* all neighbors of *Trial* that are not *Alive*. If the neighbor is in *Far*, remove it from that list and add it to the set *Close*.
3. Recompute the values of u at all *Close* neighbors of *Trial* by solving the piecewise quadratic equation according to (14).
4. Add the point *Trial* to *Alive*; remove it from *Close*.
5. Return to top of Loop.

3.2. Heap Sorts and Computational Efficiency. The key to an efficient version of the above technique lies in a fast way of locating the grid point in the narrow band with the smallest value for u . An efficient scheme for doing this is discussed in detail in [30]; here we follow that discussion.³

There are several ways to store the *Trial* elements so that one can easily find the smallest element (see, for example, [5]). In our case, imagine that we have an ordered structure of the elements in *Trial*. When a point is accepted, its neighbors are updated, and their u values may change. Thus, only a small subset of the structure must be reordered in order to regain the ordering.

³The work and contributions of Dr. Ravikanth Malladi were invaluable in the design of the initial version of the Fast Marching Method.

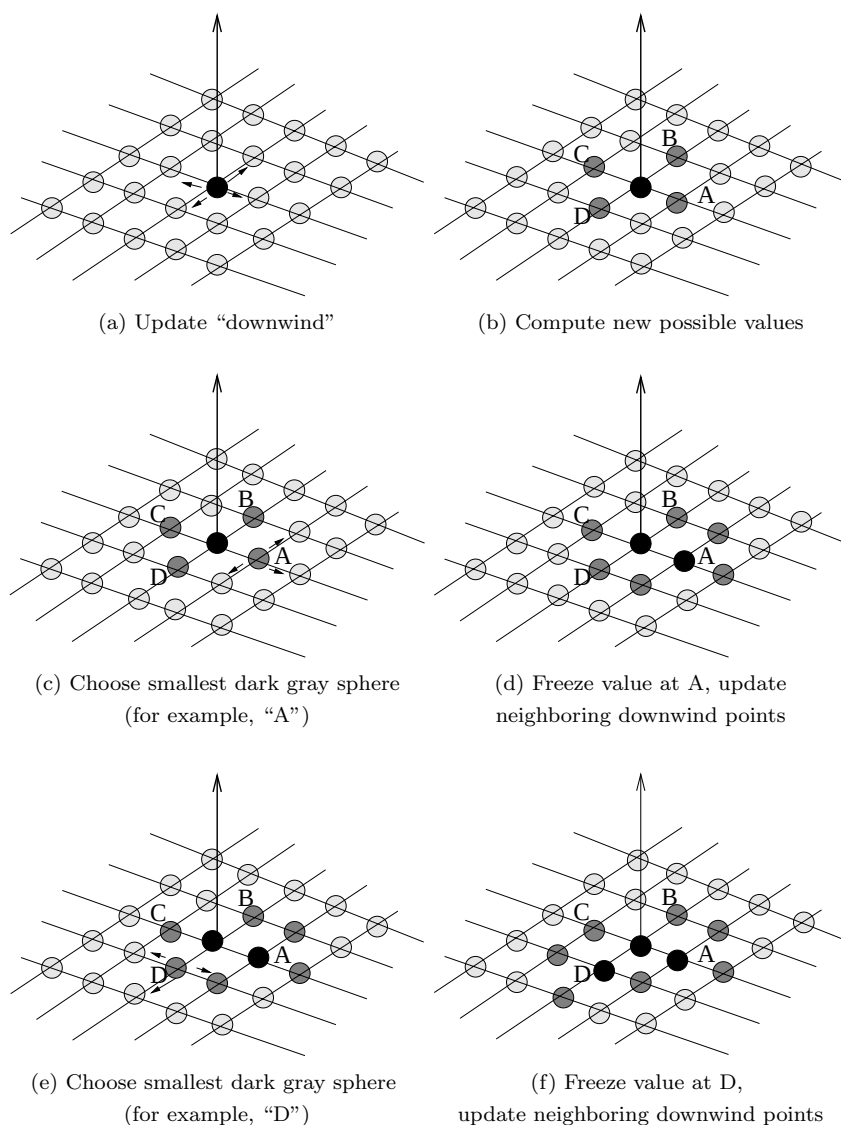


Fig. 7 Update procedure for Fast Marching Method.

This leads quite naturally to a variation on a heap algorithm (see Sedgewick [22]) with back pointers to store the u values. Specifically, we use a min-heap data structure. In an abstract sense, a min-heap is a "complete binary tree" with a property that the value at any given node is less than or equal to the values at its children. In practice, it is more efficient to represent a heap sequentially as an array by storing a node at location k and its children at locations $2k$ and $2k+1$. From this definition, the parent of a given node at k is located at $k/2$. Therefore, the root that contains the smallest element is stored at location $k=1$ in the array. Finding the parent or children of a given element are simple array accesses that take $O(1)$ time.

The values of u are stored, together with the indices that give their location in the grid structure. The marching algorithm works by first looking for the smallest element

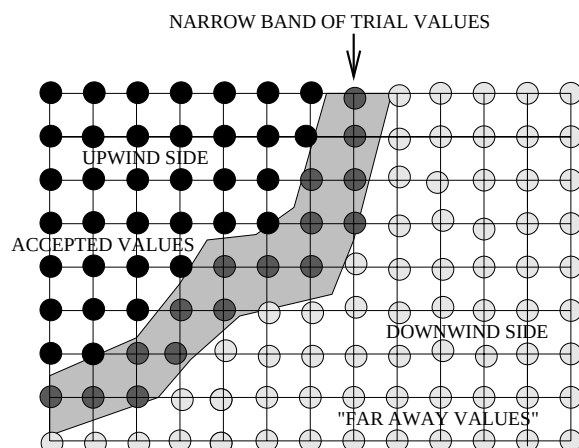


Fig. 8 Upwind construction of accepted values.

in the *NarrowBand*; this **FindSmallest** operation involves deleting the root and one sweep of **DownHeap** to ensure that the remaining elements satisfy the heap property. The algorithm proceeds by tagging the neighboring points that are not *Alive*. The *FarAway* neighbors are added to the heap using an **Insert** operation and values at the remaining points are updated using (14). **Insert** works by increasing the heap size by one and trickling the new element upward to its correct location using an **UpHeap** operation. Last, to ensure that the updated u values do not violate the heap property, we need to perform an **UpHeap** operation starting at that location and proceeding up the tree.

The **DownHeap** and **UpHeap** operations (in the worst case) carry an element all the way from root to bottom or vice versa. Therefore, this takes $O(\log N)$ time, assuming there are N elements in the heap. It is important to note that the heap, which is a complete binary tree, is always guaranteed to remain balanced. All that remains is the operation of searching for the *NarrowBand* neighbors of the smallest element in the heap. This can be done in time $O(1)$ by maintaining back pointers from the grid to the heap array. Without the back pointers, the search takes time $O(N)$ in the worst case. As an example, Figure 9 shows a typical heap structure and an **UpHeap** operation after the element at location (2, 7) gets updated from 3.1 to 2.0.

Since the total work in changing the value of one element of the heap and bubbling its value upwards is $O(\log M)$, where M is the size of the heap, this produces a total operation count of $M \log M$ for the Fast Marching Method on a grid of M total points. Thus, if we imagine a three-dimensional grid of N points in each direction, the Fast Marching Method reduces the total operation count from N^4 to $N^3 \log N$; essentially, each grid point is visited once to compute its arrival time value. For more details, see [27, 28, 30, and 17].

4. Related Algorithms.

4.1. Network Path Algorithms. The Fast Marching Method is reminiscent of Dijkstra's algorithm [10] (see also [5, 22]), which is a method for finding the shortest path on a network with prescribed weights between each link. As illustration, imagine one is given a rectangular network with equal unit cost of entering each link (see Figure 10).

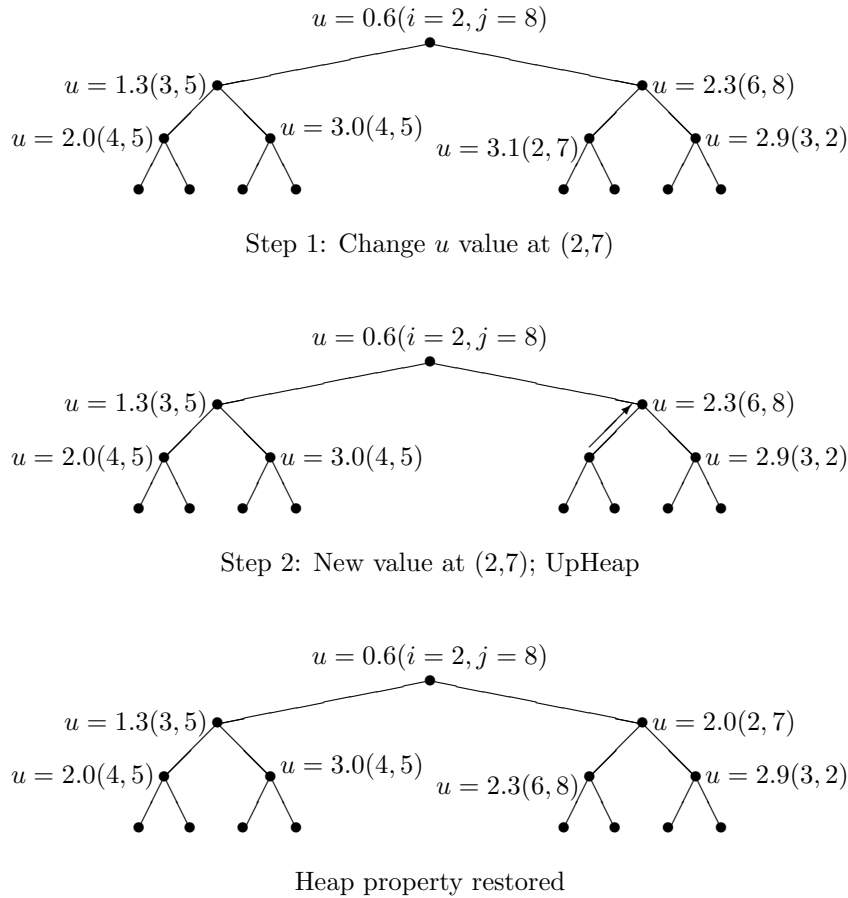


Fig. 9 Heap structure and UpHeap+ operation.

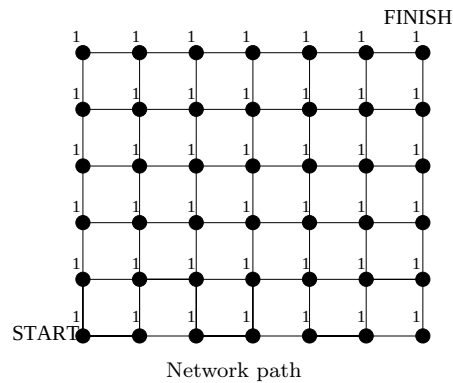


Fig. 10 Inconsistent vs. consistent algorithms.

Dijkstra's method starts at the "START" point and expands outward, visiting each node and keeping a running cost. The "front" is advanced by looking for the node reached with the smallest current cost, and then advancing to neighbors. In this sense, the method is similar to the Fast Marching Method; suitably programmed

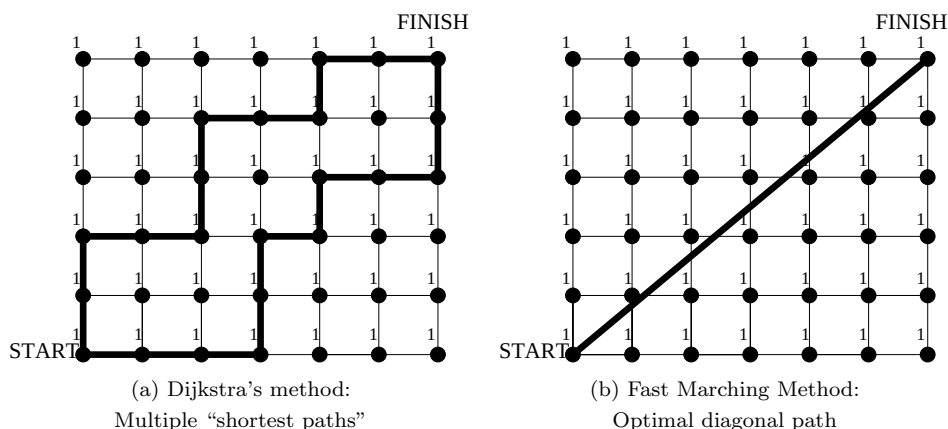


Fig. 11 Inconsistent vs. consistent algorithms.

by means of a heap as described above, the method is an $O(N \log N)$ method of computing the cost of going from point A to point B along the network.

However, if the two points are positioned relative to the network so that the optimal path is a straight-line diagonal between the two, such a graph search algorithm cannot distinguish between the various Manhattan graphs of equal cost connecting points A and B (see Figure 11a). The actual problem one wants to approximate is the solution of the *continuous* problem, rather than the network solution. Thus, such algorithms are said to be *inconsistent* with the underlying continuous problem; refinement of the network will not produce a solution that converges to the correct diagonal shortest path. While it is possible to rectify this problem by adding diagonal links to the graph, the Fast Marching Method provides an approach that directly approximates the solution of the underlying partial differential equation through consistent numerical approximations, that is, one that selects the correct diagonal with relatively few nodes shown in Figure 11b.

4.2. Optimal Orderings. A different way to look at the Fast Marching Method is by returning to the iterative loop given in (15). The Fast Marching Method is a reordering of the points so that the inner loop becomes a backsolve, and hence no outer loop iteration is necessary. The price of computing the correct reordering, done while the loop is in progress, is $O(\log N)$, which is what is required to reorder the heap once values are updated.

4.3. Relation to Level Set Methods. A companion technique, which also grew out of the work on the theory and numerics of curve/surface evolution developed in [24], is known as the level set method and also requires the notions of entropy conditions and viscosity solutions in order to produce stable and accurate numerical schemes. Level set methods, described by Osher and Sethian [18], track the motion of interfaces propagating under complex speed laws and share with Fast Marching Methods the ability to track fronts that change topology, break, and merge. They handle more complex speed motions, including speeds that depend on local curvature, and can track fronts that move forwards and backwards. However, they are considerably more computationally expensive than Fast Marching Methods.

To delineate the differences between the two techniques, imagine a closed curve Γ in the plane propagating normal to itself with speed $\bar{F}$. Furthermore, assume that

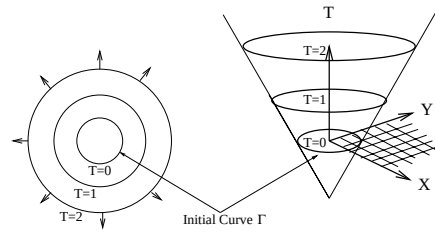


Fig. 12 Transformation of front motion into boundary value problem.

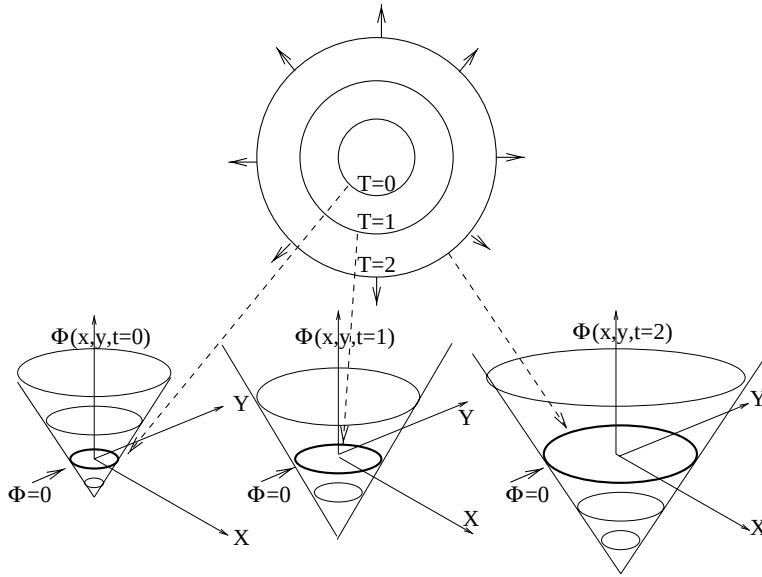


Fig. 13 Transformation of front motion into an initial value problem.

$\bar{F} > 0$, hence the front always moves “outwards.” One way to characterize the position of this expanding front is to compute the arrival time $T(x, y)$ of the front as it crosses each point (x, y) , as shown in Figure 12. Since the speed $\bar{F}$ is inversely proportional to the gradient, we must have

$$(16) \quad |\nabla T| \bar{F} = 1, \quad T = 0 \text{ on } \Gamma.$$

Thus, the front motion is characterized as the solution to a boundary value problem; if the speed $\bar{F}$ depends only on position, then the equation reduces to our familiar Eikonal equation, with $\bar{F} = 1/F$.

Conversely, suppose we embed the initial position of the front as the zero level set of a function ϕ in one higher dimension. The level set method identifies the evolution of this level set function ϕ with the propagation of the front itself through the time-dependent initial value problem

$$(17) \quad \Phi_t + \bar{F} |\nabla \Phi| = 0.$$

This equation describes the time evolution of the level set function Φ in such a way that the zero level set of this evolving function is always identified with the propagating interface; see Figure 13.

The most obvious distinction between the two views is that the initial value level set formulation allows for both positive and negative speed functions $\bar{F}$; the front may move forwards and backwards as it evolves. The boundary value perspective is restricted to fronts that always move in the same direction, because it requires a single crossing time T at each grid point, and hence a point cannot be revisited.

Thus, we wish to solve the following problems:

Initial Value Formulation	Boundary Value Formulation
(18) $\phi_t + \bar{F} \nabla\phi = 0,$ Front = $\Gamma(t) = \{(x, y) \phi(x, y, t) = 0\},$ Applies for arbitrary $\bar{F}$.	$ \nabla T \bar{F} = 1,$ Front = $\Gamma(t) = \{(x, y) T(x, y) = t\},$ Requires $\bar{F} > 0.$

Recall that the Fast Marching Method is an efficient “adaptive” technique which drops the computational labor involved in solving the boundary value formulation from $O(N^4)$ to $O(N^3 \log N)$, assuming N points in each direction of a three-dimensional problem. Similarly, an efficient implementation of the level set method, introduced in [1] and called the *narrow band level set method*, is available. The original level set method given in [18] requires $O(N^4)$ work; here we assume roughly N time steps for the front to propagate. In comparison, the narrow band method focuses all the computational labor onto a thin band around the zero level set, thus reducing the labor to $O(N^3 k)$, where k is the width of this narrow band, providing an efficient technique for implementing level set methods.⁴

Schematically, the history of the two methods and their adaptive versions is shown in Figure 14

5. Extensions of Fast Marching Methods.

5.1. Higher-Accuracy Fast Marching Methods. As presented, the Fast Marching Method is a first-order scheme, owing to the use of a first-order approximation to the gradient, namely,

$$(19) \quad \left[\begin{array}{l} \max(D_{ijk}^{-x}u, -D_{ijk}^{+x}u, 0)^2 \\ + \max(D_{ijk}^{-y}u, -D_{ijk}^{+y}u, 0)^2 \\ + \max(D_{ijk}^{-z}u, -D_{ijk}^{+z}u, 0)^2 \end{array} \right]^{1/2} = F_{ijk}.$$

We now discuss higher order Fast Marching Methods.

We begin by noting a slightly different implementation of the Fast Marching Method.⁵ When values in *Trial* (which are the tentative values in the heap) are being recomputed, use only *Known* values in the computation. Consider the second-order backward approximation to the first derivative u_x (see [9]), given by

$$u_x \approx \frac{3u_i - 4u_{i-1} + u_{i-2}}{2\Delta x},$$

⁴At first glance, the computational savings in the Fast Marching Method may not be evident on the basis of these operation counts. However, two additional advantages provide the largest computational savings. First, because the level set method is solving a time-dependent problem, time step restrictions in terms of Courant–Friedrichs–Levy (CFL) conditions based on the speed $\bar{F}$ determine the number of steps required to evolve a front; in contrast, the Fast Marching Method has no such restriction. The speed $\bar{F}$ of the front is irrelevant to the efficiency of the method. Second, the number of elements in the heap depends on the length of the front; in most cases, this length is small enough that, for all practical purposes, heap lookup is very fast.

⁵This variation was suggested by D. Adalsteinsson.

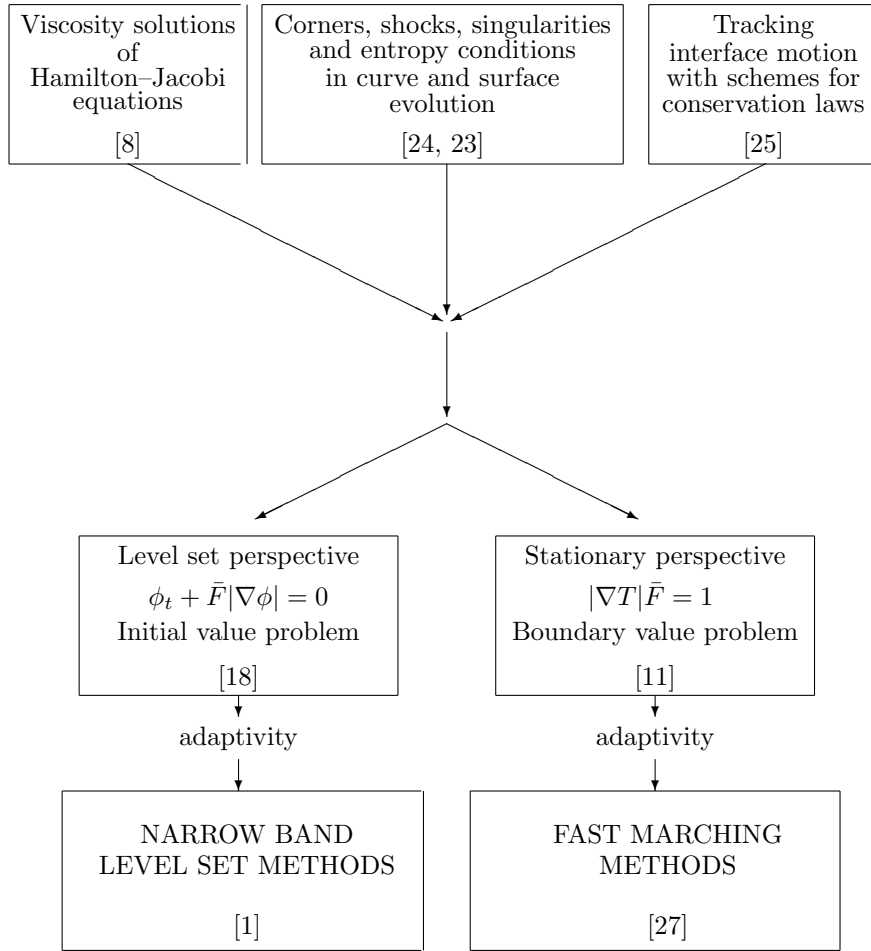


Fig. 14 Evolution of theory and algorithms for interface propagations.

which may be compactly written as

$$u_x \approx D^{-x}u + \frac{\Delta x}{2}(D^{-x})^2u.$$

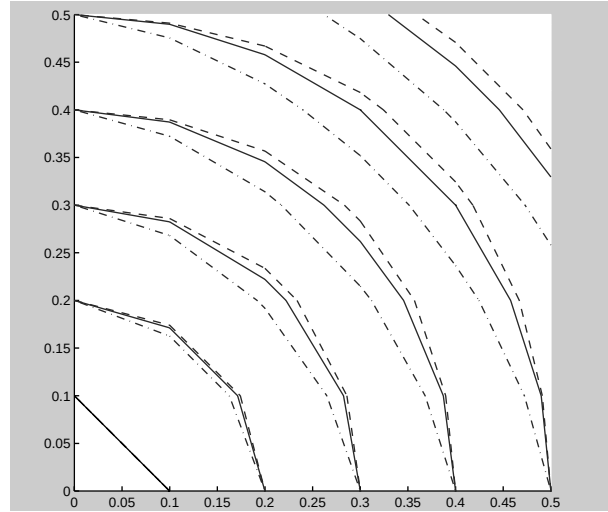
A similar expression holds for the forward difference, namely,

$$u_x \approx D^{+x}u - \frac{\Delta x}{2}(D^{+x})^2u.$$

Now consider the switch functions defined by (the expressions are similar in y and z)

$$\text{switch}_{ijk}^{-x} = \begin{bmatrix} 1 & \text{if } u_{i-2,j,k} \text{ and } u_{i-1,j,k} \text{ are known and } u_{i-2,j,k} \leq u_{i-1,j,k} \\ 0 & \text{otherwise} \end{bmatrix},$$

$$\text{switch}_{ijk}^{+x} = \begin{bmatrix} 1 & \text{if } u_{i+2,j,k} \text{ and } u_{i+1,j,k} \text{ are known and } u_{i+2,j,k} \leq u_{i+1,j,k} \\ 0 & \text{otherwise} \end{bmatrix}.$$



Equidistant points from origin: Crude 6×6 grid
Solid = exact; short dashed/dotted = 1st order; long dashed = 2nd order.

Grid	L_2 error 1st order	L_2 error 2nd order	L_∞ error 1st order	L_∞ error 2nd order
21^3	0.019790	0.007203	0.034042	0.012921
51^3	0.009409	0.002410	0.017063	0.004325
101^3	0.004610	0.000461	0.008570	0.000734
151^3	0.002289	0.000071	0.004325	0.000205

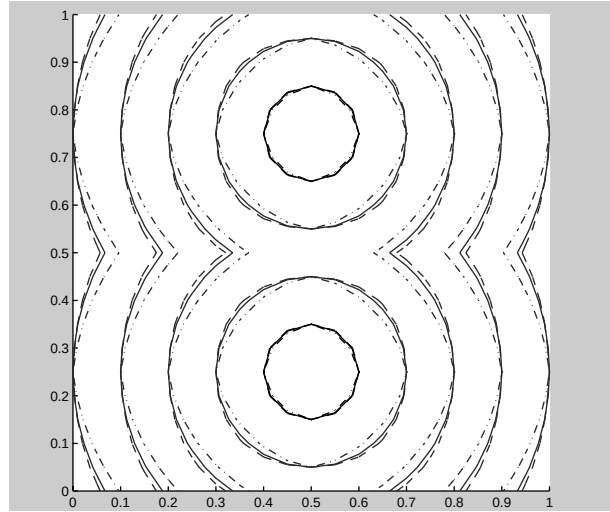
Fig. 15 First- and second-order computations of distance from origin.

We can then use these operators in the Fast Marching Method, namely,

$$\begin{aligned}
 & \left[\begin{aligned} & \max \left[\left[D_{ijk}^{-x} u + \text{switch}_{ijk}^{-x} \frac{\Delta x}{2} (D_{ijk}^{-x})^2 u \right], - \left[D_{ijk}^{+x} u - \text{switch}_{ijk}^{+x} \frac{\Delta x}{2} (D_{ijk}^{+x})^2 u \right], 0 \right]^2 \\ & + \\ & \max \left[\left[D_{ijk}^{-y} u + \text{switch}_{ijk}^{-y} \frac{\Delta y}{2} (D_{ijk}^{-y})^2 u \right], - \left[D_{ijk}^{+y} u - \text{switch}_{ijk}^{+y} \frac{\Delta y}{2} (D_{ijk}^{+y})^2 u \right], 0 \right]^2 \\ & + \\ & \max \left[\left[D_{ijk}^{-z} u + \text{switch}_{ijk}^{-z} \frac{\Delta z}{2} (D_{ijk}^{-z})^2 u \right], - \left[D_{ijk}^{+z} u - \text{switch}_{ijk}^{+z} \frac{\Delta z}{2} (D_{ijk}^{+z})^2 u \right], 0 \right]^2 \end{aligned} \right]^{1/2} \\
 & = F_{ijk}.
 \end{aligned}
 \tag{20}$$

This scheme attempts to use a second-order one-sided upwind stencil whenever points are available, but reverts to a first-order scheme in the other cases. We make two observations. First, in order to start the scheme, we must use a higher order scheme to produce accurate values in a band around the boundary values. Second, we have chosen a very “conservative” higher order version of our Fast Marching Method; it is possible to devise versions that invoke the first-order scheme less often.

5.1.1. Accuracy and Order of this Scheme. The question of whether the “second-order” Fast Marching Method is really second-order depends on how often the switches evaluate to zero and on how the number of those points where a first-order method is invoked changes as the mesh is refined. In many simulations, the number of points



Equidistant points from two points: Crude 21×21 grid
 Solid = exact; short dashed/dotted = 1st order; long dashed = 2nd order.

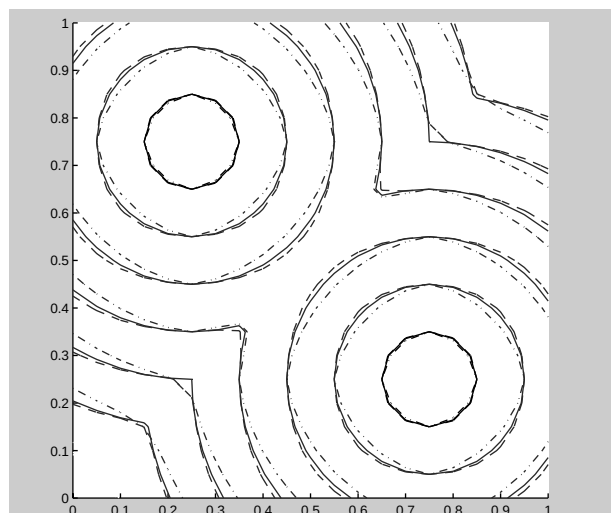
Grid	L_2 error 1st order	L_2 error 2nd order	L_∞ error 1st order	L_∞ error 2nd order
21^3	0.0096	0.00500	0.0185	0.01047
41^3	0.0051	0.00119	0.0101	0.00230
81^3	0.0028	0.00028	0.0056	0.00055
161^3	0.0014	0.00006	0.0028	0.00014

Fig. 16 First- and second-order computations of distance from two points.

where the first-order stencil must be chosen is relatively small, and the fraction of such points among the total often decreases as the mesh is refined. In these cases, the error is experimentally observed to be considerably reduced using this approach. The advantages of using second-order operators wherever possible can be seen most clearly along diagonals, where the first-order approximation to the gradient is significantly worse than the second-order version. One can also use third- and higher order one-sided differences; the degree to which these yield higher accuracy will again depend on how often the first-order scheme is invoked.

5.1.2. Tests of Scheme. Next, we examine the accuracy of the Fast Marching Method. The “second-order” scheme is defined as the one that uses second-order one-sided operators whenever possible. Figure 15 shows sets of points equidistant from the origin, computed using (i) the exact distance, (ii) the first-order scheme, and (iii) the “second-order” scheme on an extremely crude 6×6 grid. As can be seen, the “second-order” method is considerably better. Figure 15 also gives the error associated with computing the distance function from a single point located at the origin.

Next, we repeat this test, but use the Fast Marching Method to compute the distance from two points, one located at $(0.5, 0.25)$ and the other at $(0.5, 0.75)$. Here, the solution is nondifferentiable, and the viscosity solution is chosen. Once again, the “second-order” method is considerably more accurate. Figure 16 shows a calculation on a rough 21×21 grid; the table shows the error under mesh refinement.



Equidistant points from two points: Crude 21×21 grid
 Solid = exact; short dashed/dotted = 1st order; long dashed = 2nd order.

Grid	L_2 error 1st order	L_2 error 2nd order	L_∞ error 1st order	L_∞ error 2nd order
21^3	.01410	.005329	.02534	.013800
41^3	.00646	.001832	.01228	.005449
81^3	.00311	.000409	.00607	.002538
161^3	.0015	.000113	.00303	.001346

Fig. 17 First- and second-order computations of distance from two points.

Finally, we repeat this test using a grid rotation and compute the distance from two points, one located at $(0.25, 0.75)$ and the other at $(0.75, 0.25)$. The L_2 error is still second-order convergent for the second-order scheme, whereas the L_∞ error is first-order. This is to be expected: first-order error occurs along the shock lines, which, due to the grid alignment, occur between grid points. Figure 17 shows a calculation on a rough 21×21 grid; the table shows the error under mesh refinement.

5.2. Nonuniform Orthogonal Grids. The implementation of Fast Marching Methods on nonuniform orthogonal meshes is straightforward. Nonuniform Cartesian meshes are handled by the difference operators. Cylindrical and spherical coordinate meshes require the standard altered expressions for the gradient in the upwind directions. Since the use of these altered expressions does not affect the flow of the Fast Marching Method, the actual algorithm remains unchanged. Only the update formula is changed.

5.3. Triangulated Unstructured Mesh Formulation. There are a variety of reasons to develop a Fast Marching Method for triangulated domains. First, triangulated grids may be constructed to fit material or fluid boundaries. For example, problems in photolithographic development in nonrectangular regions can require a triangulated meshing so that the nonetchable material boundaries are accurately represented. Second, surfaces are often described by triangulated patches, and hence this discretization is natural. In [13], a Fast Marching Method on triangulated unstructured meshes was introduced, using the unstructured mesh methodology for level set methods developed by Barth and Sethian [2], and applied to the problem of constructing geodesic

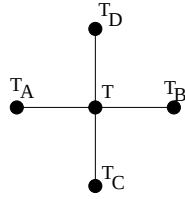


Fig. 18 Construction of upwind solution on orthogonal mesh.

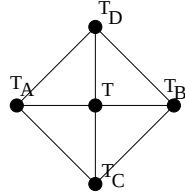


Fig. 19 Simple triangulation for building a monotone update operator.

shortest paths on manifolds. Various examples were given there, and we follow that work closely in this presentation. For further details, see [13].

5.3.1. The Update Procedure. As an introduction, recall the update procedure in the Fast Marching Method for readjusting the values of neighbors that are downwind to known points. Our goal is to solve the Eikonal equation $|\nabla T| = F$. This is the procedure by which new trial values are created for T in the heap of nearby points. Imagine a uniform square grid and suppose that the goal is to update the value of T at the center point (i, j) . We label the values of u at the surrounding grid points $T_A = T_{i-1,j}$, $T_B = T_{i+1,j}$, $T_C = T_{i,j-1}$, and $T_D = T_{i,j+1}$ (see Figure 18). Some of the values may be infinite, corresponding to *Far* values.

Standing at the center point, the Fast Marching Method attempts to solve the quadratic equation given by each quadrant. For example, we refer to possible contributors A and C . Without loss of generality, there are two cases:

1. If only T_A is known, then we find the solution $T_A < T$ to the “quadratic” equation

$$(T - T_A)^2 = h^2 F_{ij}^2,$$

where h is the uniform grid spacing.

2. If T_A and T_C are known, then we take the real solution to the quadratic equation

$$(T - T_A)^2 + (T - T_C)^2 = h^2 F_{ij}^2.$$

For each possible up-down or left-right pair, we construct all possible real solutions; we then accept as the updated point the one that produces the smallest value of T . This is the Fast Marching Method described in [27] and [28].

5.3.2. A Scheme for a Particular Triangulated Domain. We now extend this method to triangulated domains. In order to do so, we shall build a monotone update procedure on the triangulated mesh. As motivation, we consider the obvious triangulation of a square grid in the plane. Imagine the triangulation given in Figure 19.

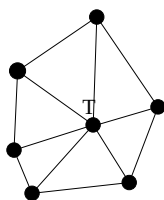


Fig. 20 Acute triangulation around center grid point.

If we consider the known values T_A and T_C , and look at the triangle formed by the points that hold T_A , T_C , and T , we can easily write down the equation of the plane determined by the known values T_A and T_C , as well as the unknown value T , namely (with h as the side length),

$$\left[\frac{T - T_A}{h} \right] x + \left[\frac{T - T_C}{h} \right] y + T = z.$$

Computing the gradient, we then want to select a value of T such that

$$\left[\frac{T - T_A}{h} \right]^2 + \left[\frac{T - T_C}{h} \right]^2 = F_{ij}^2.$$

In other words, we are lifting the plane by a value T at the center point i, j in order to have a gradient magnitude equal to $1/F$. We note that this is the exact same construction as the one produced by the “orthogonal” construction. Note also that in this case the gradient vector, with origin at the center point, always points into the triangle from which it is updated. This is not necessarily the case for an arbitrary acute triangle. In order to establish monotonicity we will need to verify this condition.

The inability to solve this quadratic corresponds to an inability to tilt at an appropriate angle, and the requirement that the solution T be greater than the contributors means that the solution is always constructed in an upwind manner. Using the same update rules as before, and the heap structure to maintain a list of *Trial* points, this provides a method for executing the Fast Marching Method on this simple triangulation.

5.3.3. Fast Marching Methods on Triangulated Domains. Following the construction of unstructured mesh upwind approximations to the gradient [2], we now extend this idea to an arbitrary triangulation.

Acute Triangulations. We start with an acute triangulation and consider the triangulation around the grid point given in Figure 20. A large number of triangles may share the center vertex. Our procedure, motivated by the simple triangulation in the previous section, is to compute a possible value for T from each triangle that includes the center point as a vertex. Since several triangles can produce admissible values for T , we must select an appropriate value. There are several possibilities. We chose the one that produces the smallest new value for T ; this will correspond to an algorithm similar to the one used on the triangular mesh. More elaborate upwind constructions on triangulated meshes are given in [2].

Consider the nonobtuse triangle ABC in which the point to update is C . Assume that $T(B) > T(A)$. We first verify that the update is from within the triangle; i.e., the altitude h should be inside the nonobtuse triangle CBD (see Figure 21). This

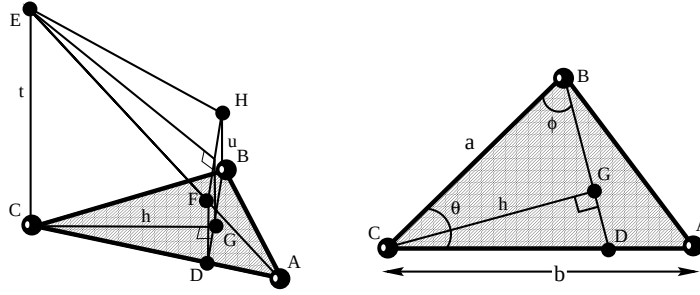


Fig. 21 Given the triangle ABC , such that $u = u(B) - u(A)$, find $u(C) = u(A) + t$ such that $(t - u)/h = F$. Left: A perspective view of the triangle stencil supporting the $u()$ values that form a lifted plane with a gradient magnitude equal to F . Right: The trigonometry on the plane defined by the triangle stencil.

means that we search for $t = EC$ such that

$$\frac{t - u}{h} = F.$$

Denote $a = BC$ and $b = AC$; we have, by similarity, that $t/b = DF/AD = u/AD$, thus $CD = b - AD = b - bu/t = b(t - u)/u$. Next, by the law of cosines, $BD^2 = a^2 + CD^2 - 2aCD \cos \theta$, and by the law of sines, $\sin \phi = \frac{CD}{BD} \sin \theta$. Now, using the right angle triangle CBG , we have

$$h = a \sin \phi = a \frac{CD}{BD} \sin \theta = \frac{aCD \sin \theta}{\sqrt{a^2 + CD^2 - 2aCD \cos \theta}}.$$

This yields a quadratic equation for t :

$$(a^2 + b^2 - 2ab \cos \theta)t^2 + 2bu(a \cos \theta - b)t + b^2(u^2 - F^2 a^2 \sin^2 \theta) = 0.$$

The solution t must satisfy $u < t$ and should be updated from within the triangle, namely,

$$(21) \quad a \cos \theta < \frac{b(t - u)}{t} < \frac{a}{\cos \theta}.$$

Thus, the update procedure is given as follows:

$$\text{If } u < t \text{ and } a \cos \theta < \frac{b(t - u)}{t} < \frac{a}{\cos \theta},$$

$$\text{then } T(C) = \min\{T(C), t + T(A)\};$$

$$\text{else } T(C) = \min\{T(C), bF + T(A), cF + T(B)\}.$$

This is the scheme to extend the Fast Marching Method to acute triangulated domains given in [13].

Extension to General Triangulations. So far we have required an acute triangulation. This is so that any front entering the side of a triangle will have two points to provide values before the third is computed. In other words, for monotonicity, we

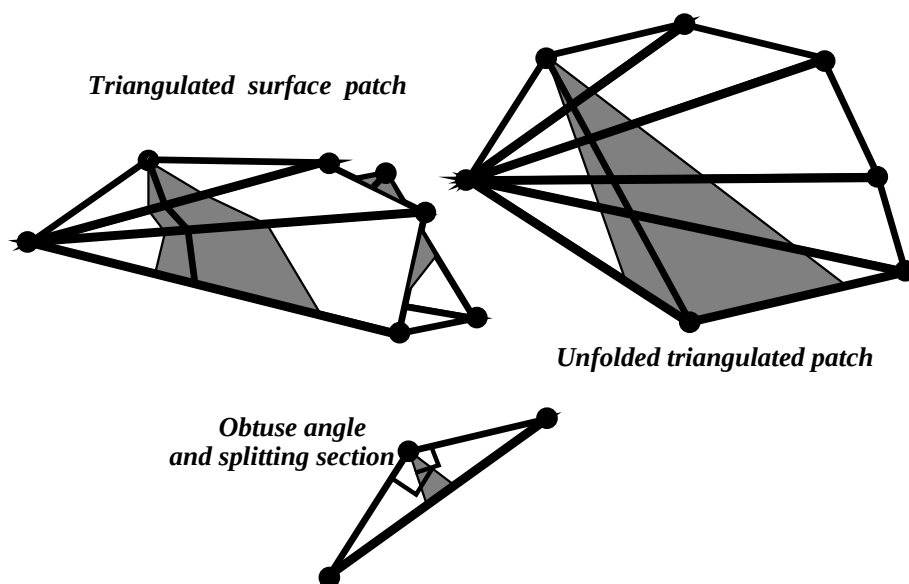


Fig. 22 Upper left: A triangulated surface patch. Bottom: The initialization of the construction for the splitting section. Upper right: The unfolded patch and the splitting section expansion up to the first vertex B , and the virtual edge connecting the two vertices AB .

restrict the update to come from within the triangle; i.e., the gradient of the solution at a grid point should point into the triangle from which it is updated. The most straightforward way to enforce this requirement is to build a nonobtuse triangulation, thus making sure that the grid captures all incoming fronts.

One approach to handling nonacute triangulations, which we now describe, is to build numerical support locally at obtuse angles by splitting these angles in a special way. An obtuse angle at vertex A can be updated by its neighboring points in a consistent way only at a limited section of upcoming fronts. Connecting the vertex to any point in this section splits the obtuse angle into two acute ones.

The difficulty with this approach is that we need to reach back and use more triangles than simply the one containing the point to be updated. These other triangles might not be coplanar, since the unstructured mesh may lie on a convoluted surface. The idea is to extend this section by recursively unfolding the adjacent triangle(s), until a new vertex B is included in the extended section. Then the vertices are connected by a virtual directional edge from B to A (i.e., A may be updated by B). The length of the edge AB is equal to the distance between A and B on the unfolded triangle(s) plane (see Figure 22).

Finally, we note that a complexity analysis for this unfolding procedure given in [13] shows that the $O(N \log N)$ operation count is maintained. We refer the interested reader there for further details. Further discussion of triangulated Fast Marching Methods, higher order versions, and extensions to the basic techniques may be found in [33].

6. Applications. In this section, we provide a collection of examples to demonstrate the applicability of Fast Marching Methods.

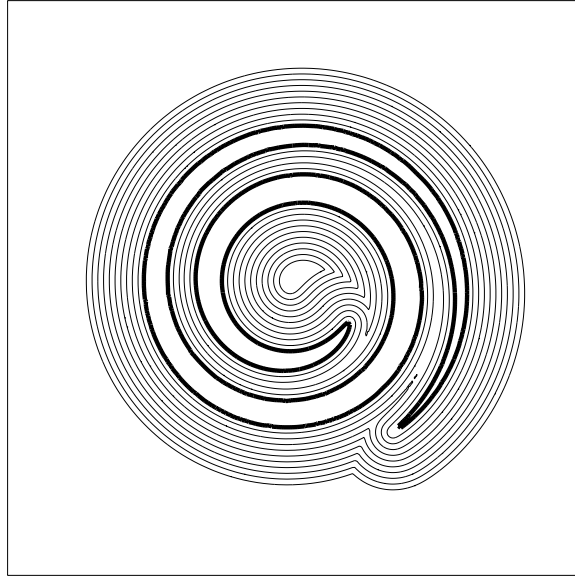


Fig. 23 *Shape offsetting of spiral.*

6.1. Shape Offsetting. The first example, borrowed from [30], is the straightforward problem of shape offsetting. Given a closed shape in two or three dimensions, one wants to compute the offset, obtained by propagating the boundary in its normal direction with constant unit speed. All that is required is to compute the distance to the boundary and to plot the level curves.

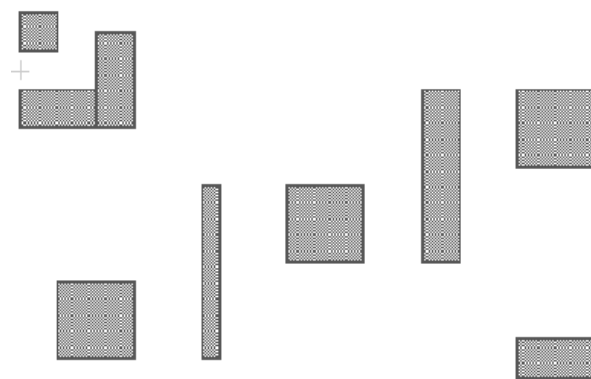
Here, we work directly with the Eikonal equation. Since the speed is $F = 1$, we must then solve

$$(22) \quad |\nabla T| = 1,$$

where the boundary condition is $T = 0$ on and inside the given shape. In Figure 23, the boundary is represented as a dark heavy line, and we show the larger shape-offsets, obtained using the Fast Marching Method.

6.2. Photolithographic Development. One aspect of the manufacturing of microchips is the process of *lithographic development*. In this process, the resist properties of a material are altered through exposure to a beam that has been partially blocked by a pattern mask. The material is then “developed,” which means the material with less resistivity is etched away. The development process reduces to that of following an interface propagating downwards in three dimensions, where the speed in the normal direction is given as a supplied rate function at each point. The speed $F = F(x, y, z)$ depends only on position; however, it may change extremely rapidly. The goal in lithographic development is to track this evolving front. In order to develop realistic structures in three-dimensional development profiles, a grid of size $300 \times 300 \times 100$ is not unreasonable; hence a fast algorithm is of considerable value in the development step.

As illustration (see [30]), a rate function calculated using the three-dimensional exposure and postexposure bake modules of TMA’s Depict 4.0 [36] is coupled to the Fast Marching Method. Figure 24a shows the top view of a mask placed on the



(a) Masking pattern



(b) Lithographic development: View from below

Fig. 24 *Lithographic development using Fast Marching Method.*

board; the dark areas correspond to areas that are exposed to light. The presence of such factors as standing waves in the etching profile depends on issues such as the reflectivity of the surface. In Figure 24b, a view of the developed profile is shown from underneath; the etching of the holes and the presence of standing waves can be seen easily. For further results, see [29].

6.3. Seismic Travel Times. Next, we apply Fast Marching Methods to problems involving the imaging of geophysical data sets. In [32], Sethian and Popovici used the Fast Marching Method to rapidly construct first arrival times in seismic analysis, and

then coupled this work to prestack migration. Here, we summarize that work. For further details, see [32].

Three-dimensional prestack migration of surface seismic data is used to image the earth's subsurface when complex geological structures and velocity fields are present. The most commonly used imaging techniques applied to three-dimensional prestack surveys are methods based on the Kirchhoff integral, because of its flexibility in imaging irregularly sampled data and its relative computational efficiency. To perform Kirchhoff migration, one approximately solves the wave equation with a boundary integral method. The reflectivity at every point of the earth's interior is computed by summing the recorded data on multidimensional surfaces; the shapes of the summation surfaces and the summation weights are computed from the Green's functions of the single scattering wave-propagation experiment (see, for example, [21]).

6.3.1. Background Equations. In more detail, the essence of three-dimensional prestack migration (see, for example, [19]) is expressed by the following integral equation:

$$\text{Image}(\mathbf{x}) = \int \int_{\mathbf{x}_s} \int_{\mathbf{x}_r} G(\mathbf{x}_s, \mathbf{x}, \omega) G(\mathbf{x}, \mathbf{x}_r, \omega) \text{Data}(\mathbf{x}_s, \mathbf{x}_r, \omega) d\mathbf{x}_r d\mathbf{x}_s d\omega,$$

where $\mathbf{x}$ is the image output location, $\mathbf{x}_s$ and $\mathbf{x}_r$ are the data source and receiver coordinates, and ω is angular frequency. The Green's functions $G(\mathbf{x}_s, \mathbf{x}, \omega)$ and $G(\mathbf{x}, \mathbf{x}_r, \omega)$ parameterize propagation from source to image point and from image point to receiver, respectively. In most implementations, the calculation is often done instead in the time domain, and can be expressed as the summation

$$\text{Image}(\mathbf{x}) = \sum_{\mathbf{x}_s} \sum_{\mathbf{x}_r} A_s A_r \text{Input}(\mathbf{x}_s, \mathbf{x}_r, t_s + t_r),$$

where Input is a filtered version of the input data, and the Green's functions are parameterized by the amplitudes A_s and A_r and travel times t_s and t_r .

For three-dimensional prestack Kirchhoff depth migration, the Green's functions are represented by five-dimensional tables; these tables are functions of the source/receiver surface locations (x, y) and of the reflector position (x, y, z) in the earth's interior. This Green's function parameterization is usually based on the assumption of acoustic propagation. This Kirchhoff prestack migration process consists of two stages. First, travel time tables are computed and stored. Second, the migrated image is formed by convolving the prestack data with migration operators derived from the travel time tables. Both phases present challenges from the perspective of both the geophysical accuracy and the computer implementation.

The key element of three-dimensional prestack Kirchhoff depth migration is the calculation of travel time tables used to parameterize the asymptotic Green's functions. An efficient travel time calculation method is required to generate the five-dimensional travel time tables needed for three-dimensional Kirchhoff migration.⁶ Also, since depth migration problems are generally applied in areas of complex velocity structure, the travel time calculation method must be robust. Computing three-dimensional Green's function tables over a 100×100 kilometer area (about 430 marine blocks), with sources positioned every 200 meters, requires 1 terabyte of travel time volumes. Thus, speed is an important issue. Finite difference approximations

⁶The Green's function can be reconstructed from travel time tables that describe travel times from all surface points (x, y) to all subsurface locations (x, y, z) ; thus the tables are five-dimensional.

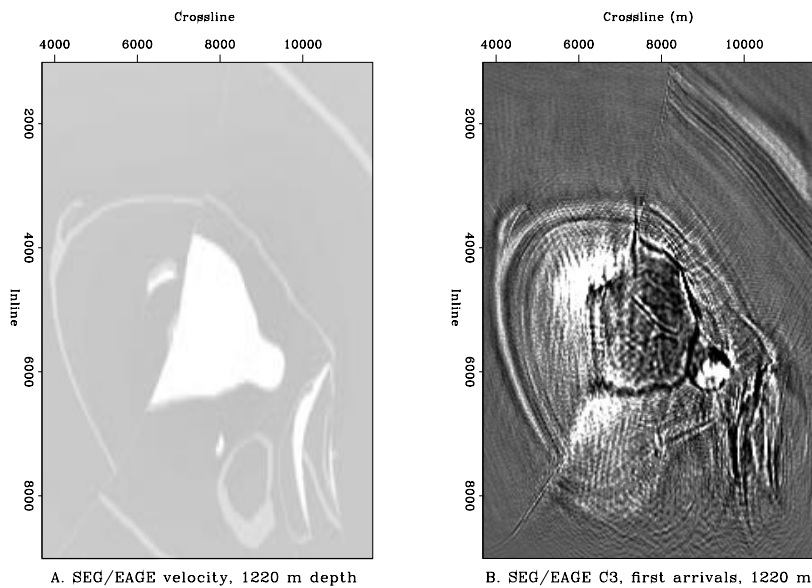


Fig. 25 *Velocity model and and migrated image.*

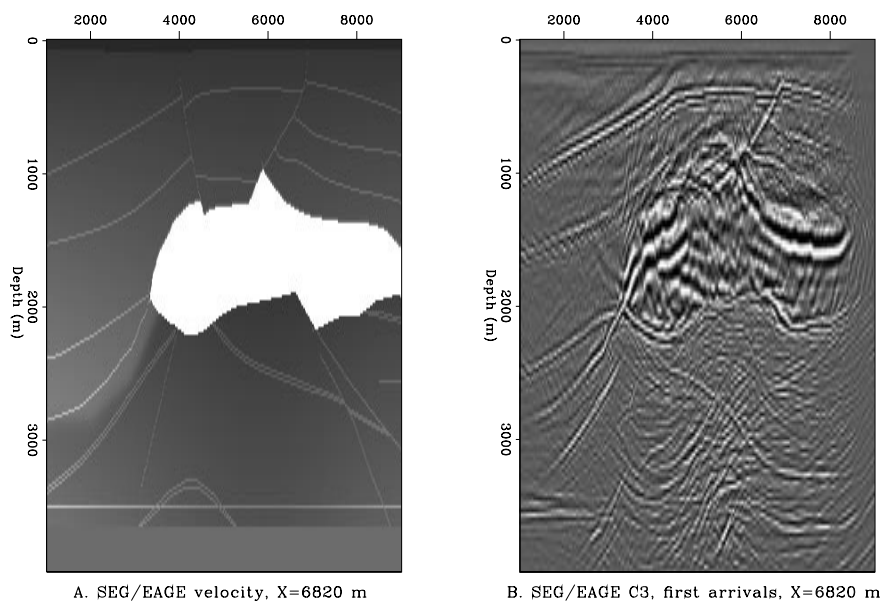


Fig. 26 *Velocity model and migrated image.*

to travel time computations include the work of Vidale [38] and van Trier and Symes [37]. Because of their speed and robustness and the fact that they are unconditionally stable, Fast Marching Methods offer attractive methods of computing travel times.

6.3.2. Migration Using the Fast Marching Method. Figures 25 and 26 show slices through the three-dimensional velocity and corresponding structural images obtained from migration on prestack data obtained from a given data set. The left side

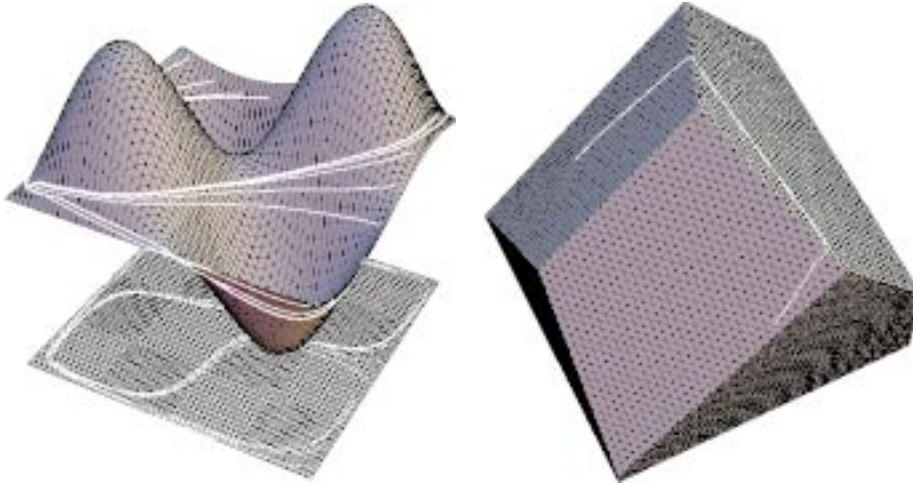


Fig. 27 *Computing minimal weighted geodesics on triangulated surfaces.*

of Figure 25 shows a depth slice through a velocity cube at a depth of 1220 meters; the right side shows the corresponding migrated image slice. The salt-sediment interface and the semicircular fault cutting through the salt body are imaged with high resolution. Figure 26 compares the velocity model on the left with the corresponding migrated line on the right, for a different slice. The sediment images are imaged at the correct locations, together with the salt body borders. The areas with lesser quality are under the salt, most probably as a result of multiple reflected arrivals at this spot from the water bottom and intrasalt reflections, and close to the left side of the top of the salt, most probably because of the use of first arrivals in the Fast Marching Method. For further results and discussion of other issues, see [32] and [35].

6.4. An Optimal Algorithm for Computing Geodesic Paths on Surfaces. Suppose we try to compute the shortest path between two points on a given surface. This minimal geodesic can be obtained from a problem in front propagation as follows. As discussed in [13], one uses the triangulated Fast Marching Method to solve the straightforward Eikonal equation $|\nabla T| = 1$ on a triangulated approximation to the manifold. One can then backtrace on the surface itself, solving the ordinary differential equation

$$\frac{dX(s)}{ds} = -\nabla T,$$

where $X(s)$ traces out the geodesic path. We use a second-order Heun's integration method on the triangulated surface with a switch to a first-order scheme at sonic points in the gradient.

Here, we show two examples taken from [13]. Figure 27 presents a perspective view of the triangulation and shortest paths for two surfaces. The left side of the figure shows shortest paths on regular triangulation of the surface given by the function $z(x, y) = 0.45 \sin(2\pi x) \sin(2\pi y)$ on $[0, 1] \times [0, 1]$, for a grid size of 50×50 . The minimal geodesics are painted on the triangulated surface and projected to the $x - y$ plane. The right side of the figure gives a polyhedron example, in which a different

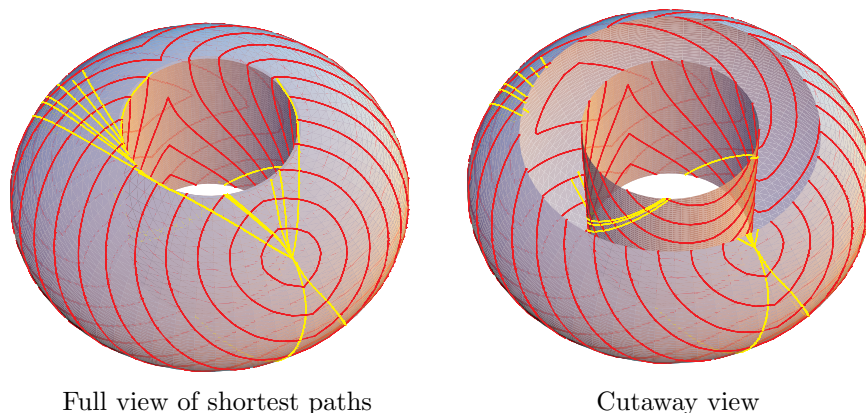


Fig. 28 Shortest paths on a bead (a genus-1 two-dimensional manifold).

speed function F was assigned to each side, causing a Snell's law effect along the edges. The speed F is 2 at the close side with the start point, 1 at the second top side, and 4 at the side of the destination point.

Next, we show a computation that illustrates the performance on manifolds with an underlying nonacute (obtuse) triangulation. Figure 28 shows the computation of minimal geodesics on a torus, in which some shortest paths in fact cut through the middle. The equidistance curves are shown, as well as the shortest paths. We note that the computational complexity is unchanged; we have given an $O(N \log N)$ algorithm for constructing geodesic paths on triangulated surfaces.⁷ For details, see [13].

6.5. Optimal Path Planning.

6.5.1. Statement of Problem. The application of Fast Marching Methods to problems in path planning was first developed in [12]; here we summarize some of those results.

Given a cost function $F(x_1, x_2, \dots, x_n)$, and a starting point A in R^n , one goal in path planning is to construct the path $\gamma(\tau) : [0, \infty) \rightarrow R^n$ from A to any point B in R^n that minimizes the integral

$$(23) \quad \int_{A=\gamma(0)}^{B=\gamma(L)} F(\gamma(\tau)) d\tau,$$

where τ is the arclength parameterization of γ ; namely, $|\gamma_\tau| = 1$, and L is the total length of γ .

More specifically, in two dimensions, suppose we are given the cost function $F(x, y)$ and a starting point A . Let $T(x, y)$ be the minimal cost required to travel from A to the point (x, y) , that is,

$$(24) \quad T(x, y) = \min_{\gamma} \int_A^{(x,y)} F(\gamma(\tau)) d\tau,$$

⁷It is important to be clear about what is being claimed. We are *not* finding the exact shortest path for a given triangulation of a surface. The solution path using the above algorithm is the shortest path within an error that depends on the size of the triangulation. It is an *approximate* shortest path, constructed in $O(N \log N)$ steps, where N is the number of triangles used to tessellate the surface. As the triangulation is refined, this path converges to the exact shortest path.

with $|\gamma_\tau| = 1$. The level set $T(x, y) = C$ is the set of all points in R^2 that can be reached with minimal cost C , and the minimal cost paths are orthogonal to the level curves; hence we have

$$(25) \quad |\nabla T| = F(x, y).$$

Obviously, the Fast Marching Method can be used to solve this Eikonal equation to produce $T(x, y)$ in all of R^2 . Then, given a point B in R^2 , explicit construction of the shortest path comes through backpropagation from B to A via the solution of the ordinary differential equation

$$(26) \quad X_t = -\nabla T \quad \text{given } X(0) = B,$$

until we reach the starting point A .

6.5.2. Results. For an illustration of these techniques, consider the problem of navigation with constraints and rotation. Instead of a robot as a single point, we imagine a two-dimensional rectangle with a given width and length. Thus, the initial position A of the robot in configuration space is specified by the position of the center of the rectangle, plus an angle θ between 0 and 2π . The final configuration B is similarly specified, and the goal is to construct the optimal path from A to B .

In the absence of obstacles, a completely straightforward application of the Fast Marching Method is possible: one discretizes the configuration space into a three-dimensional grid consisting of a discretized mesh for both R^2 and θ employing periodic boundary conditions for θ . Thus, we solve the Eikonal equation

$$(27) \quad [u_x^2 + u_y^2 + u_\theta^2]^{1/2} = 1.$$

In the presence of obstacles, we take the following approach; see [14]. Rather than maneuver an oddly shaped robot, we instead consider the robot as a point, and, for every discretized angle θ_i , alter the shape of the obstacles corresponding to that angle. To do this for all obstacles at each angle requires morphological shape operations consisting of dilations and translations; these too may be done using the Fast Marching Method; see [12]. In Figure 29 we show several examples of a two-dimensional robot with rotational angle navigating in two-dimensional space around obstacles. For further details about applications of Fast Marching Methods to path planning and robotic navigation, see [12] and [31].

6.6. Visibility Calculations and Reflected Bounces. Our last application is the evaluation of visibility. By visibility, we mean the determination of whether two points in a domain have an unobstructed straight-line view of each other. Such issues are important in several areas:

- *Semiconductor manufacturing.* One factor that influences the etching and deposition process is whether a point on an evolving profile is visible from other points on the surface. As an example, imagine an etching beam that dislodges particles as it strikes the surface; these dislodged particles leave the interface and are then redeposited somewhere else on the profile. Thus, the total flux of particles into any point on the front depends on computing the amount received from all other points on the front. There are various models that describe how these dislodged particles are ejected, including specular reflection (the angle of incidence equals the angle of refraction) and luminescent reflection (the angle at which a dislodged particle leaves the surface is uniformly distributed among all possible angles). In most cases, visibility

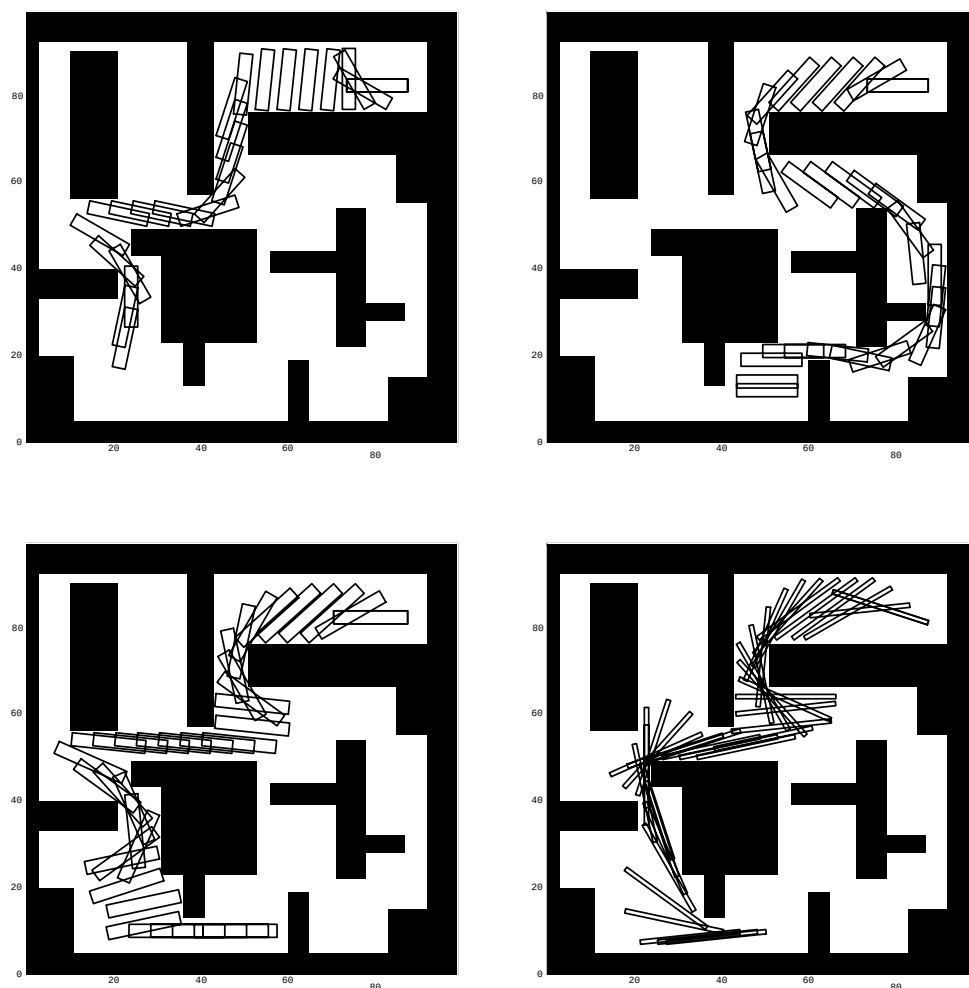


Fig. 29 *Two-dimensional navigation around obstacles with rotation.*

comes into play; the amount of material received at a point from dislodged particles emanating from another point on the surface is zero if there is no direct line of sight between the two points, meaning that some other part of the profile blocks the incoming flux.

- *Ray tracing and scene rendering:* In computer graphics, ray tracing is often used to accurately render a complex scene. A scene can have many obstacles, corners, and occluded objects; the fundamental algorithm requires tracing a beam from a source to a final location, determining which points are directly visible.
- *Shadowing:* In other computer graphics and optics problems, given a source location, one wants to quickly locate the set of all points shadowed by a given collection of obstacles.

Fast Marching Methods can aid in computing the visibility in these situations. Specifically, we give algorithms for the following:

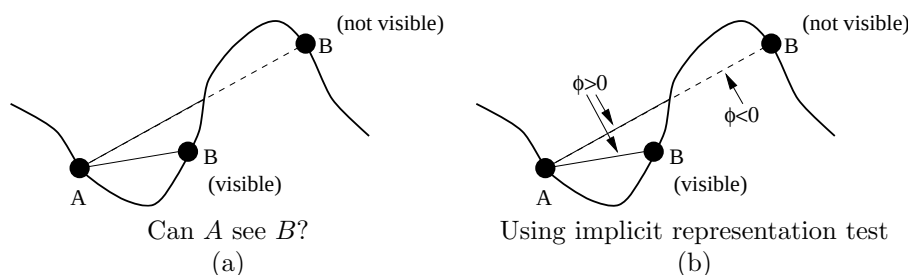


Fig. 30 Visibility test between A and B .

1. *Profile visibility*: Given a profile (that is, a curve in two dimensions or surface in three dimensions), we can determine whether the straight-line path between two points is blocked by the profile itself.
 2. *Scene visibility*: Given a domain filled with obstacles in either two or three dimensions, we can determine if two points in the domain can see each other directly.
 3. *Shadowing*: Given a source and a domain filled with obstacles, we can provide a fast algorithm to determine those points that are shadowed by the obstacles.
- We now discuss these algorithms in some detail.

6.6.1. Profile Visibility. Suppose we are given a profile and two points A and B ; we wish to determine if there is a direct line of sight between A and B (see Figure 30a).

A brute force approach, while easy to program, is costly. Suppose that there are N points on the curve. By checking all segments between the two points, it costs $O(N)$ to determine if A can see B . Thus, checking whether each pair of points is mutually visible is $O(N^3)$. Of course, clever programming can make this much faster, including quadtree representations and judicious choices of how to proceed through the list of segments.

A fast algorithm comes from exploiting an implicit representation of the interface. Given the profile, we use the Fast Marching Method to determine the signed distance T from the profile. Then we need only check to see if $T(x) > 0$ for all points x on the segment between A and B . If T is always positive, then the two points are visible, since the front cannot intersect the segment. We can perform this test of whether $T(x) > 0$ along the segment as follows. First, pick the midpoint of the segment and evaluate $T(x)$ by interpolation from the underlying grid; the value indicates the distance to the closest point on the front. One can now move that distance along the line segment in both directions and then query again. Repeating this process until one either reaches both A and B (in which case the two points were mutually visible) or reaches a point where $\phi < 0$ (in which case the two points were not mutually visible) terminates the algorithm; see Figure 30b.

6.6.2. Scene Visibility. Now imagine a domain filled with obstacles, as in Figure 31. The goal is to determine whether or not two points A and B are visible to each other.

We may proceed in a manner similar to the above and use the Fast Marching Method to determine scene visibility. First, we compute the distance function $T(x, y)$ from all the obstacles. We then can test the visibility of any two points in the scene by the above algorithm, which queries whether $T(x, y) < 0$ at any point on the line connecting A and B .

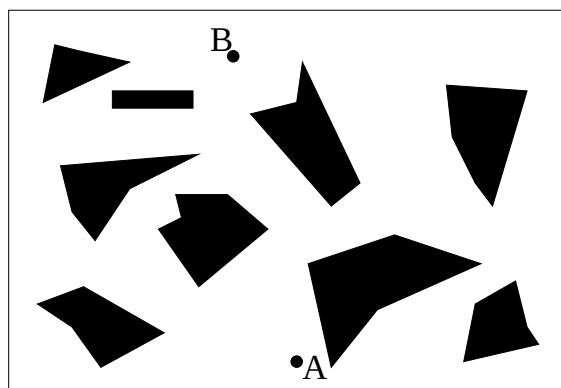


Fig. 31 Scene visibility.

How does this approach compare to a more direct algorithm which, given the line between two points, checks to see if it intersects any of the obstacles? The answer depends on the relative density of obstacles in the domain. If a large fraction of the domain is filled with obstacles, then the Fast Marching Method computes the distance function in the empty spaces quickly. On the other hand, the direct testing approach of checking all obstacles against a given segment connecting two points can be time consuming. In this case, the Fast Marching approach is viable. Conversely, if there are only a few, small obstacles in the domain, then the price of gridding the domain to construct the solution to the Eikonal equation may not be competitive with a direct testing approach.

6.6.3. Shadowing. Finally, we give a fast algorithm for approximately computing the shadow region behind obstacles created from a point source. Thus, given a collection of obstacles, we wish to find all points in a domain directly visible from a source at A . For a small number of obstacles, there are standard computer graphics techniques that typically perform comparisons with each obstacle. As the number of obstacles increases, so can the complexity of such algorithms.

A different approach is as follows. Given a source at A , imagine the solution to two separate Eikonal equations:

$$(28) \quad |\nabla T_{\text{no-obstacles}}| = 1, \quad |\nabla T_{\text{obstacles}}| = F(x, y),$$

where the right-hand side in the *obstacles* case is set to ∞ inside the obstacles and 1 otherwise. We solve each of these problems separately, using the Fast Marching Method, and then compare the solutions. If the obstacle blocks the source A from the point (x, y) , then the first arrival time should be larger in the *obstacles* case. Thus, if $T_{\text{no-obstacles}}(x, y) = T_{\text{obstacles}}(x, y)$, then the point (x, y) is visible from the source; if, on the other hand, $T_{\text{no-obstacles}}(x, y) < T_{\text{obstacles}}(x, y)$, then the point is not visible. In practice, due to numerical error in the scheme, we need to use a threshold on the difference; that is, we check that

$$T_{\text{no-obstacles}}(x, y) + \text{threshold} < T_{\text{obstacles}}(x, y).$$

The threshold is chosen as a function of the mesh size h . An equally effective approach comes from comparing the results of running the method with grid size h and $h/2$.

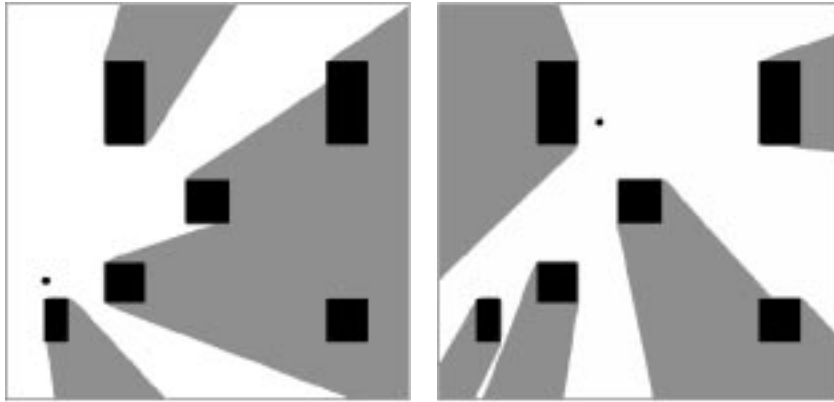


Fig. 32 *Shadow zone created by obstacles.*

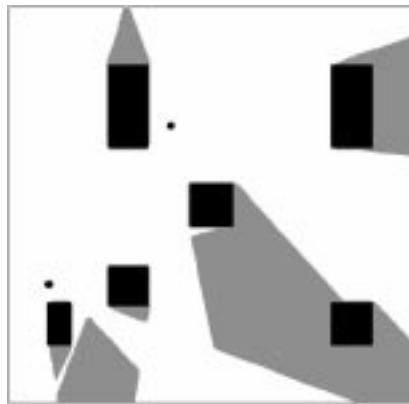


Fig. 33 *Shadow zone for two sources.*

We note two advantages with this approach. First, the computational speed is independent of the number of obstacles, and second, the technique is unchanged in three dimensions and higher. In Figure 32, we show the shadow zone created around six obstacles for two different placements of the sources.

The technique may be easily extended to multiple sources by running each source separately and then comparing the separate arrival times (see Figure 33). For further discussion of visibility and related issues, see [31].

7. Additional Results. The range of applications for Fast Marching Methods is large and currently a field of active interest. These methods are in use in such areas as computer graphics, medical imaging, antenna and wave problems, crack dynamics, and flame propagation. They provide very fast ways to compute extension velocities that are critical to the efficient implementation of level set methods, and have found their way into morphological operations, Boolean operations on shapes, and implicit surface CAD/CAM descriptors. We refer the interested reader to [31] for a much more detailed discussion, as well as many additional applications.

Acknowledgments. All calculations were performed at the University of California, Berkeley, and the Lawrence Berkeley National Laboratory, Berkeley, CA. We would like to thank D. Adalsteinsson, R. Kimmel, R. Malladi, and M. Popovici for many useful discussions. A web page devoted to Fast Marching Methods may be found at http://math.berkeley.edu/~sethian/level_set.html

REFERENCES

- [1] D. ADALSTEINSSON AND J.A. SETHIAN, *A fast level set method for propagating interfaces*, J. Comput. Phys., 118 (1995), pp. 269–277.
- [2] T.J. BARTH AND J.A. SETHIAN, *Numerical schemes for the Hamilton–Jacobi and level set equations on triangulated domains*, J. Comput. Phys., 145 (1998), pp. 1–40.
- [3] D.L. CHOPP, *Computing minimal surfaces via level set curvature flow*, J. Comput. Phys., 106 (1993), pp. 77–91.
- [4] P. COLELLA AND E.G. PUCKETT, *Modern Numerical Methods for Fluid Flow*, Lecture Notes, Department of Mechanical Engineering, Univ. of California, Berkeley, 1994.
- [5] T. CORMEN, C. LEISERSON, AND R. RIVEST, *Introduction to Algorithms*, McGraw-Hill, New York, 1990.
- [6] M.G. CRANDALL, L.C. EVANS, AND P.-L. LIONS, , *Some properties of viscosity solutions of Hamilton–Jacobi equations*, Trans. Amer. Math. Soc., 282 (1984), pp. 487–502.
- [7] M.G. CRANDALL, H. ISHII, AND P.-L. LIONS, *User’s guide to viscosity solutions of second order partial differential equations*, Bull. Amer. Math. Soc., 27 (1992), pp. 1–67.
- [8] M.G. CRANDALL AND P.-L. LIONS, *Viscosity solutions of Hamilton–Jacobi equations*, Trans. Amer. Math. Soc., 277 (1983), pp. 1–43.
- [9] G. DAHQUIST AND A. BJÖRK, *Numerical Methods*, Prentice-Hall, Englewood Cliffs, NJ, 1974.
- [10] E.W. DIJKSTRA, *A note on two problems in connection with graphs*, Numer. Math., 1 (1959), pp. 269–271.
- [11] P. GARABEDIAN, *Partial Differential Equations*, Wiley, New York, 1964.
- [12] R. KIMMEL AND J. A. SETHIAN, *Fast Marching Methods for Robotic Navigation with Constraints*, Center for Pure and Applied Mathematics Report, Univ. of California, Berkeley, May 1996; Internat. J. Robotics Research, 1998, submitted.
- [13] R. KIMMEL AND J.A. SETHIAN, *Fast marching methods on triangulated domains*, Proc. Nat. Acad. Sci., 95 (1998), pp. 8341–8435.
- [14] J.C. LATOMBE, *Robot motion planning*, Kluwer, Boston, MA, 1991.
- [15] R.J. LEVEQUE, *Numerical Methods for Conservation Laws*, Birkhauser, Basel, 1992.
- [16] R. MALLADI, J.A. SETHIAN, AND B.C. VEMURI, *Evolutionary fronts for topology-independent shape modeling and recovery*, in Proceedings of Third European Conference on Computer Vision, Stockholm, Sweden, Lecture Notes in Comput. Sci. 800, Springer-Verlag, New York, 1994, pp. 3–13.
- [17] R. MALLADI AND J.A. SETHIAN, *An $O(N \log N)$ algorithm for shape modeling*, Proc. Nat. Acad. Sci., 93 (1996), pp. 9389–9392.
- [18] S. OSHER AND J.A. SETHIAN, *Fronts propagating with curvature dependent speed: Algorithms based on Hamilton–Jacobi formulation*, J. Comput. Phys., 79 (1988), pp. 12–49.
- [19] M. POPOVICI, *Prestack migration by split-step DSR*, Geophysics, 61 (1996), pp. 1412–16.
- [20] E. ROUY AND A. TOURIN, *A viscosity solutions approach to shape-from-shading*, SIAM J. Numer. Anal., 29 (1992), pp. 867–884.
- [21] W.A. SCHNEIDER, JR., *Robust and efficient upwind finite-difference traveltime calculations in three dimensions*, Geophysics, 60 (1995), pp. 1108–1117.
- [22] R. SEDGEWICK, *Algorithms*, Addison-Wesley, Reading, MA, 1988.
- [23] J.A. SETHIAN, *An Analysis of Flame Propagation*, Ph.D. Thesis, Dept. of Mathematics, Univ. of California, Berkeley, 1982.
- [24] J.A. SETHIAN, *Curvature and the evolution of fronts*, Comm. Math. Phys., 101 (1985), pp. 487–499.
- [25] J.A. SETHIAN, *Numerical methods for propagating fronts*, in Variational Methods for Free Surface Interfaces, P. Concus and R. Finn, eds., Springer-Verlag, New York, 1987.
- [26] J.A. SETHIAN, *Numerical algorithms for propagating interfaces: Hamilton–Jacobi equations and conservation laws*, J. Differential Geom., 31 (1990), pp. 131–161.
- [27] J.A. SETHIAN, *A marching level set method for monotonically advancing fronts*, Proc. Nat. Acad. Sci., 93 (1996), pp. 1591–1595.
- [28] J.A. SETHIAN, *Applications of level set methods for propagating interfaces*, Acta Numerica, 1996, pp. 309–395.

- [29] J.A. SETHIAN, *Fast marching level set methods for three-dimensional photolithography development*, in Proceedings, SPIE 1996 International Symposium on Microlithography, Santa Clara, CA, March 1996.
- [30] J.A. SETHIAN, *Level Set Methods: Evolving Interfaces in Geometry, Fluid Mechanics, Computer Vision and Material Science*, Cambridge University Press, Cambridge, UK, 1996.
- [31] J.A. SETHIAN, *Level Set Methods and Fast Marching Methods: Evolving Interfaces in Computational Geometry, Fluid Mechanics, Computer Vision and Materials Sciences*, Cambridge University Press, Cambridge, UK, 1998.
- [32] J.A. SETHIAN AND A.M. POPOVICI, *Three dimensional traveltimes computation using the Fast Marching Method*, Geophysics, March-April, 1999.
- [33] J.A. SETHIAN AND A. VLADIMIRSKY, *Extensions to Triangulated Fast Marching Methods*, to be submitted for publication, 1998.
- [34] G.A. SOD, *Numerical Methods in Fluid Dynamics*, Cambridge University Press, Cambridge, UK, 1985.
- [35] 3DGeo CORPORATION, *Computing and Imaging Using Fast Marching Methods*, 3DGeo Corporation, Internal Report, June 1998, Sunnyvale, CA.
- [36] TECHNOLOGY MODELING ASSOCIATES, *Three-Dimensional Photolithography Simulation with Depict 4.0*, Technology Modeling Associates, Internal Documentation, January 1996, Sunnyvale, CA..
- [37] J. VAN TRIER AND W.W. SYMES, *Upwind finite-difference calculations of traveltimes*, Geophysics, 56 (1991), pp. 812–821.
- [38] J. VIDAŁE, *Finite-difference calculation of travel times*, Bull. Seismal Soc. Amer., 78 (1988), pp. 2062–2076.
- [39] J. VIDAŁE, *Finite-difference calculation of traveltimes in three dimensions*, Geophysics, 55 (1990), pp. 521–526.